

회원가입

≡ API	/user/register
🕒 메소드	POST
≡ 보안	bcrypt
☀ 상태	완료
≡ 설명	사용자가 새로운 계정을 등록합니다.

1. API 개요 (사용자 회원가입)

- API 경로: /user/register
- 메서드: POST
- 설명: 사용자가 새로운 계정을 등록합니다.

2. Request

요청 형식

```
{
  "userId": "String",      // 사용자 ID (필수)
  "userPassword": "String", // 비밀번호 (필수, 암호화 저장)
  "userEmail": "String",   // 이메일 주소 (필수, 고유)
  "userName": "String",   // 사용자 이름 (필수)
  "userPhoneNum": "String", // 전화번호 (필수)
  "userAddress": "String"  // 주소 (필수)
}
```

요청 필드 설명

필드	타입	필수 여부	설명
userId	String	필수	사용자 ID (필수)
userPassword	String	필수	사용자 비밀번호 (암호화 저장)
userEmail	String	필수	사용자 이메일 (고유)
userName	String	필수	사용자 이름
userPhoneNum	String	필수	사용자 전화번호
userAddress	String	필수	사용자 주소

3. Response

성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "User registered successfully"
}
```

실패 시

```
{
  "resultCode": "FAILURE",
  "message": "User registration failed",
  "errors": [
    {
      "field": "String",    // 오류가 발생한 필드 (예: "userId", "userEmail")
      "message": "String"  // 해당 필드의 오류 메시지 (예: "User ID already exists")
    }
  ]
}
```

4. 설명

Request 설명

1. `userId`: 필수, 사용자가 지정한 ID, 중복 불가.
2. `userPassword`: 필수, 비밀번호는 암호화(bcrypt).
3. `userEmail`: 필수, 유효한 이메일 주소, 중복 불가.
4. `userName`: 필수, 사용자 이름.
5. `userPhoneNum`: 필수, 전화번호.
6. `userAddress`: 필수, 사용자 주소.

Response 설명

1. 성공 (`SUCCESS`)
 - `resultCode`: "SUCCESS"
 - `message`: "User registered successfully"
2. 실패 (`FAILURE`)
 - `resultCode`: "FAILURE"
 - `message`: "User registration failed"
 - `errors`: 오류가 발생한 필드 및 메시지를 포함한 리스트.

5. 예시

성공 요청/응답

- Request:

```
{
  "userPK": "001",
  "userId": "newUser123",
  "userPassword": "securePassword123",
  "userEmail": "user@example.com",
  "userName": "John Doe",
  "userPhoneNum": "123-456-7890",
  "userAddress": "123 Main St, City, Country"
}
```

- Response:

```
{
  "resultCode": "SUCCESS",
  "message": "User registered successfully"
}
```

실패 요청/응답

- **Request:**

```
{
  "userId": "existingUser",
  "userPassword": "password123",
  "userEmail": "",
  "userName": "John Doe",
  "userPhoneNum": "123-456-7890",
  "userAddress": "123 Main St, City, Country"
}
```

- **Response:**

```
{
  "resultCode": "FAILURE",
  "message": "User registration failed",
  "errors": [
    {
      "field": "userId",
      "message": "User ID already exists"
    },
    {
      "field": "userEmail",
      "message": "Email is required"
    }
  ]
}
```

로그인

≡ API	/user/login
🕒 메소드	POST
≡ 보안	bcrypt
⚡ 상태	완료
≡ 설명	사용자가 계정 정보를 이용하여 로그인합니다.

1. API 개요 (사용자 로그인)

- API 경로: /user/login
- 메서드: POST
- 설명: 사용자가 계정 정보를 이용하여 로그인합니다.
- 인증 방식: 로그인 성공 시 JWT 토큰 발급

2. Request

요청 형식

```
{
  "userId": "String",      // 사용자 ID (필수)
  "password": "String"     // 비밀번호 (필수)
}
```

요청 필드 설명

필드	타입	필수 여부	설명
userId	String	✅ 필수	사용자 ID
password	String	✅ 필수	사용자 비밀번호

3. Response

✅ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Login successful",
  "data": {
    "userPk": 101,          // 사용자의 PK (DB에 저장된 고유 ID)
    "userId": "validUser123",
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
  }
}
```

❌ 실패 시

```
{
  "resultCode": "FAILURE",
  "message": "Login failed",
}
```



```

"errors": [
  {
    "field": "userId",
    "message": "User ID does not exist"
  }
]
}

```

4. 설명

Request 설명

- `userId`: 필수, 사용자의 고유 ID.
- `password`: 필수, 비밀번호.

Response 설명

1. 성공 (`SUCCESS`)

- `resultCode`: `"SUCCESS"`
- `message`: `"Login successful"`
- `data`:
 - `userPk`: DB에서 저장된 사용자의 PK (고유 ID, `INT`)
 - `userId`: 로그인된 사용자 ID
 - `token`: JWT 인증 토큰

2. 실패 (`FAILURE`)

- `resultCode`: `"FAILURE"`
- `message`: `"Login failed"`
- `errors`: 오류가 발생한 필드 및 메시지를 포함한 리스트.

5. 예시

✅ 성공 요청/응답

• Request

```

{
  "userId": "validUser123",
  "password": "securePassword123"
}

```

• Response

```

{
  "resultCode": "SUCCESS",
  "message": "Login successful",
  "data": {
    "userPk": 101,
    "userId": "validUser123",
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
  }
}

```

```
}  
}
```

❌ 실패 요청/응답

1 사용자가 존재하지 않는 경우

• Request

```
{  
  "userId": "invalidUser",  
  "password": "wrongPassword123"  
}
```

• Response

```
{  
  "resultCode": "FAILURE",  
  "message": "Login failed",  
  "errors": [  
    {  
      "field": "userId",  
      "message": "User ID does not exist"  
    }  
  ]  
}
```

2 비밀번호가 틀린 경우

• Request

```
{  
  "userId": "validUser123",  
  "password": "wrongPassword"  
}
```

• Response

```
{  
  "resultCode": "FAILURE",  
  "message": "Login failed",  
  "errors": [  
    {  
      "field": "password",  
      "message": "Incorrect password"  
    }  
  ]  
}
```

모듈 세트 조회

≡ API	/user/module-set/list
🕒 메소드	GET
⚡ 상태	완료
≡ 설명	사용자가 선택 가능한 모듈 세트 목록을 조회합니다.

1. API 개요

- API 경로: /user/module-set/list
- 메서드: GET
- 설명: 사용자가 선택 가능한 모듈 세트 목록을 조회합니다.
- 기능:
 - 모듈 세트 목록을 검색 및 필터링하여 반환
 - 페이지네이션 지원
 - 각 모듈 세트에 포함된 옵션 목록 제공

2. Request

요청 형식

```
{
  "page": 1,           // 페이지 번호 (선택, 기본값: 1)
  "pageSize": 10       // 페이지 크기 (선택, 기본값: 10)
}
```

요청 필드 설명

필드	타입	필수 여부	설명
page	Integer	선택	결과 페이지 번호 (기본값: 1)
pageSize	Integer	선택	한 페이지당 반환할 항목 개수 (기본값: 10)

3. Response

성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Module sets retrieved successfully",
  "data": {
    "moduleSets": [
      {
        "moduleSetId": 1,
        "moduleSetName": "캠핑카 모듈 세트",
        "totalCost": 2000,
        "imgsUrls": [
          "https://example.com/module1.jpg"
        ]
      }
    ]
  }
}
```

```

        "description": "캠핑에 적합한 세트",
        "suppliedOptions": [
            {
                "optionId": 201,
                "optionName": "배터리 팩",
                "quantity": 1
            },
            {
                "optionId": 202,
                "optionName": "냉장고",
                "quantity": 2
            }
        ]
    },
    ],
    "pagination": {
        "currentPage": 1,
        "totalPages": 3,
        "totalItems": 25,
        "pageSize": 10
    }
}

```

실패 시

```

{
    "resultCode": "FAILURE",
    "message": "No matching module sets found",
    "data": null
}

```

4. 설명

Response 필드 설명

필드	타입	설명
resultCode	String	"SUCCESS" : 성공, "FAILURE" : 실패
message	String	요청 처리 결과에 대한 설명
data	Object / null	응답 데이터 (성공 시 반환, 실패 시 null)
└ moduleSets	Array	모듈 세트 목록
└ moduleSetId	Integer	모듈 세트 ID
└ moduleSetName	String	모듈 세트 이름
└ totalCost	Decimal	모듈 세트 가격
└ imgs	Array	이미지 URL 리스트
└ description	String	모듈 세트 설명
└ suppliedOptions	Array	해당 모듈 세트에 포함된 옵션 리스트
└ optionId	Integer	옵션 ID
└ optionName	String	옵션 이름
└ quantity	Integer	포함된 옵션 개수

	pagination	Object	페이지네이션 정보
	currentPage	Integer	현재 페이지 번호
	totalPages	Integer	총 페이지 수
	totalItems	Integer	전체 항목 개수
	pageSize	Integer	한 페이지당 항목 수

5. 예시

성공 요청/응답

Request

```
{
  "page": 1,
  "pageSize": 5
}
```

Response

```
{
  "resultCode": "SUCCESS",
  "message": "Module sets retrieved successfully",
  "data": {
    "moduleSets": [
      {
        "moduleSetId": 1,
        "moduleSetName": "캠핑카 모듈 세트",
        "totalCost": 2000,
        "imgs": [
          "https://example.com/module1.jpg"
        ],
        "description": "캠핑에 적합한 세트",
        "suppliedOptions": [
          {
            "optionId": 201,
            "optionName": "배터리 팩",
            "quantity": 1
          },
          {
            "optionId": 202,
            "optionName": "냉장고",
            "quantity": 2
          }
        ]
      }
    ],
    "pagination": {
      "currentPage": 1,
      "totalPages": 1,
      "totalItems": 1,
      "pageSize": 5
    }
  }
}
```

```
}  
}
```

실패 요청/응답

Request

```
{  
  "page": 1,  
  "pageSize": 5  
}
```

Response

```
{  
  "resultCode": "FAILURE",  
  "message": "No matching module sets found",  
  "data": null}  
}
```

옵션 목록 조회

≡ API	/user/option/list
🕒 메소드	GET
☀️ 상태	완료
≡ 설명	사용자가 선택 가능한 개별 옵션 목록을 조회합니다

1. API 개요

- API 경로: /user/option/list
- 메서드: GET
- 설명: 사용자가 선택 가능한 개별 옵션 목록을 조회합니다.
- 기능:
 - 옵션 목록을 검색 및 필터링하여 반환
 - 페이지네이션 지원

2. Request

요청 형식

```
{
  "page": 1,           // 페이지 번호 (선택, 기본값: 1)
  "pageSize": 10,      // 페이지 크기 (선택, 기본값: 10)
  "search": "String"   // 옵션 검색 키워드 (선택)
}
```

요청 필드 설명

필드	타입	필수 여부	설명
page	Integer	선택	결과 페이지 번호 (기본값: 1)
pageSize	Integer	선택	한 페이지당 반환할 항목 개수 (기본값: 10)
search	String	선택	옵션 이름 검색 키워드

3. Response

성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Options retrieved successfully",
  "data": {
    "options": [
      {
        "optionId": 201,
        "optionName": "배터리 팩",
        "optionSize": "2x3x2",
        "optionCost": 500,
        "stockQuantity": 10,
        "optionType": "switch",

```

```

    "imgURLs": [
      "https://example.com/option1.jpg"
    ],
    "description": "캠핑 모듈용 배터리 팩"
  },
  {
    "optionId": 202,
    "optionName": "태양광 패널",
    "optionSize": "3x4x1",
    "optionCost": 1500,
    "stockQuantity": 5,
    "imgs": [
      "https://example.com/option2.jpg"
    ],
    "description": "모듈용 태양광 패널"
  }
],
"pagination": {
  "currentPage": 1,
  "totalPages": 3,
  "totalItems": 25,
  "pageSize": 10
}
}
}

```

실패 시

```

{
  "resultCode": "FAILURE",
  "message": "No matching options found",
  "data": null
}

```

4. 설명

Response 필드 설명

필드	타입	설명
<code>resultCode</code>	<code>String</code>	"SUCCESS": 성공, "FAILURE": 실패
<code>message</code>	<code>String</code>	요청 처리 결과에 대한 설명
<code>data</code>	<code>Object</code> / <code>null</code>	응답 데이터 (성공 시 반환, 실패 시 <code>null</code>)
<code>options</code>	<code>Array</code>	개별 옵션 목록
<code>optionId</code>	<code>Integer</code>	옵션 ID
<code>optionName</code>	<code>String</code>	옵션 이름
<code>optionSize</code>	<code>String</code>	옵션 크기
<code>optionCost</code>	<code>Decimal</code>	옵션 가격
<code>optionType</code>	<code>String</code>	옵션 타입
<code>stockQuantity</code>	<code>Integer</code>	옵션 재고 수량
<code>imgs</code>	<code>Array</code>	옵션 이미지 URL 리스트

	description	String	옵션 설명
	pagination	Object	페이지네이션 정보
	currentPage	Integer	현재 페이지 번호
	totalPages	Integer	총 페이지 수
	totalItems	Integer	전체 항목 개수
	pageSize	Integer	한 페이지당 항목 수

5. 예시

성공 요청/응답

Request

```
{
  "search": "배터리",
  "page": 1,
  "pageSize": 5
}
```

Response

```
{
  "resultCode": "SUCCESS",
  "message": "Options retrieved successfully",
  "data": {
    "options": [
      {
        "optionId": 201,
        "optionName": "배터리 팩",
        "optionSize": "2x3x2",
        "optionCost": 500,
        "optionType": "switch",
        "stockQuantity": 10,
        "imgs": [
          "https://example.com/option1.jpg"
        ],
        "description": "캠핑 모듈용 배터리 팩"
      }
    ],
    "pagination": {
      "currentPage": 1,
      "totalPages": 1,
      "totalItems": 1,
      "pageSize": 5
    }
  }
}
```

실패 요청/응답

Request

```
{
  "search": "잘못된 검색어",
  "page": 1,
  "pageSize": 5
}
```

Response

```
{
  "resultCode": "FAILURE",
  "message": "No matching options found",
  "data": null
}
```

대여 신청

≡ API	/user/rent/request
🕒 메소드	POST
≡ 보안	Bearer Token
☀ 상태	완료
≡ 설명	사용자가 모듈 세트 및 옵션을 선택하고, 대여 기간 및 목적지를 설정하여 대여 요청을 보냅니다.

1. API 개요

- API 경로: /user/rent/request
- 메서드: POST
- 설명: 사용자가 모듈 세트 및 옵션을 선택하고, 대여 기간 및 목적지를 설정하여 대여 요청을 보냅니다.
- 인증 필요: Bearer Token
- 기능:
 - 모듈 세트 선택 (필수)
 - 옵션 선택 (선택, 없을 경우 빈 배열 [])
 - 대여 기간 설정
 - 출발지 및 도착지 설정 (SLAM 기반 목적지 좌표 사용)

2. Request

요청 형식

```
{
  "userPK" : 001,           // 사용자 고유 식별 ID (필수)
  "moduleSetId": 101,       // 선택한 모듈 세트 ID (필수)
  "optionIds": [201, 202],  // 선택한 옵션 ID 리스트 (없으면 빈 배열 [])
  "rentCost": 50000,        // 대여 비용 (필수)
  "autonomousArrivalPoint": { // SLAM 기반 목적지 좌표
    "x": 12.313,
    "y": 32.3232
  },
  "autonomousDeparturePoint": { // SLAM 기반 출발지 좌표
    "x": 11.512,
    "y": 30.4531
  },
  "rentStartDate": "2025-01-15 09:00:00", // 대여 시작 날짜
  "rentEndDate": "2025-01-20 18:00:00"   // 대여 종료 날짜
}
```

요청 필드 설명

필드	타입	필수 여부	설명
userPK	Int	필수	대여를 요청하는 사용자 고유 식별 ID
moduleSetId	Int	필수	대여할 모듈 세트 ID
optionIds	Array(Int)	선택	대여 시 추가할 옵션 리스트 (없으면 빈 배열 [])

rentCost	Decimal	필수	대여 총 비용
autonomousArrivalPoint	Object	필수	자율주행 차량의 목적지 좌표 ({x: Float, y: Float})
autonomousDeparturePoint	Object	필수	자율주행 차량의 출발지 좌표 ({x: Float, y: Float})
rentStartDate	String	필수	대여 시작 날짜 (형식: YYYY-MM-DD HH:MM:SS)
rentEndDate	String	필수	대여 종료 날짜 (형식: YYYY-MM-DD HH:MM:SS)

3. Response

성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Rental request successful",
  "data": {
    "rentId": 1,           // 생성된 대여 ID
    "carNumber": "PBV-00123" // 배정된 차량 번호
  }
}
```

실패 시

```
{
  "resultCode": "FAILURE",
  "message": "Rental request failed",
  "errors": [
    {
      "field": "String",    // 오류가 발생한 필드 (예: "moduleSetId", "rentStartDate")
      "message": "String"  // 해당 필드의 오류 메시지 (예: "Invalid moduleSetId")
    }
  ]
}
```

4. 설명

Request 설명

1. **userPK** :
 - 대여를 요청하는 사용자 고유 식별 ID.
2. **moduleSetId** :
 - 대여할 모듈 세트 ID.
3. **optionIds** :
 - 대여 시 선택한 추가 옵션 ID 리스트.
 - 선택 사항이며, 제공하지 않으면 기본값으로 `null` 또는 빈 배열 `[]`.
4. **rentCost** :
 - 해당 대여 요청에 대한 총 비용.
5. **autonomousArrivalPoint** :
 - **SLAM** 시각화 지도에서 사용자가 핀을 찍은 좌표 ({x: Float, y: Float}).
6. **autonomousDeparturePoint** :

- 자율주행 차량이 출발할 좌표 ({x: Float, y: Float}).

7. `rentStartDate`, `rentEndDate` :

- 대여 시작 및 종료 시간 (YYYY-MM-DD HH:MM:SS 형식).

5. 예시

✅ 성공 요청/응답

Request

```
{
  "userPK": 001,
  "moduleSetId": 101,
  "optionIds": [201, 202],
  "rentCost": 50000,
  "autonomousArrivalPoint": {
    "x": 12.313,
    "y": 32.3232
  },
  "autonomousDeparturePoint": {
    "x": 11.512,
    "y": 30.4531
  },
  "rentStartDate": "2025-01-15 09:00:00",
  "rentEndDate": "2025-01-20 18:00:00"
}
```

Response

```
{
  "resultCode": "SUCCESS",
  "message": "Rental request successful",
  "data": {
    "rentId": 1,
    "carNumber": "PBV-00123"
  }
}
```

❌ 실패 요청/응답 (`moduleSetId` 오류)

Request

```
{
  "userPK": 001,
  "moduleSetId": 999, // 잘못된 모듈 세트 ID
  "optionIds": [],
  "rentCost": 50000,
  "autonomousArrivalPoint": {
    "x": 12.313,
    "y": 32.3232
  },
  "autonomousDeparturePoint": {
    "x": 11.512,
```

```
    "y": 30.4531
  },
  "rentStartDate": "2025-01-15 09:00:00",
  "rentEndDate": "2025-01-20 18:00:00"
}
```

Response

```
{
  "resultCode": "FAILURE",
  "message": "Rental request failed",
  "errors": [
    {
      "field": "moduleSetId",
      "message": "Module set ID not found"
    }
  ]
}
```

차량 대여 취소

≡ API	/user/rent/cancel
🕒 메소드	POST
≡ 보안	Bearer Token
☀ 상태	완료
≡ 설명	사용자가 진행 중인 대여(<code>rent_status: reserved</code> 또는 <code>in-progress</code>)를 취소합니다.

1. API 개요

- API 경로: `/user/rent/cancel`
- 메서드: `POST`
- 설명: 사용자가 진행 중인 대여(`rent_status: reserved` 또는 `in-progress`)를 취소합니다.
- 인증 필요: `Bearer Token`
- 기능:
 - 진행 중(`in-progress`) 또는 예약(`reserved`) 상태의 대여 취소 가능
 - 완료(`completed`) 상태의 대여는 취소 불가
 - 취소 후 `rent_status` 를 `canceled` 로 변경
 - 환불 정책 적용 (필요 시 `refundAmount` 포함)

2. Request

요청 형식

```
{
  "userPK": "Int",      // 사용자 고유식별 ID (필수)
  "rentId": 101,        // 대여 ID (필수)
  "reason": "String"    // 취소 사유 (선택)
}
```

요청 필드 설명

필드	타입	필수 여부	설명
<code>userPK</code>	<code>Int</code>	✅ 필수	취소 요청을 보낸 사용자 고유식별 ID
<code>rentId</code>	<code>Int</code>	✅ 필수	취소하려는 대여의 ID
<code>reason</code>	<code>String</code>	❌ 선택	취소 사유 (예: "일정 변경", "개인 사정")

3. Response

✅ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Rental cancellation successful",
  "data": {
    "rentId": 101,      // 취소된 대여 ID
  }
}
```

```
"status": "canceled", // 최종 대여 상태
"refundAmount": 1500 // 환불 금액 (있을 경우)
}
}
```

❌ 실패 시

```
{
  "resultCode": "FAILURE",
  "message": "Rental cancellation failed",
  "errors": [
    {
      "field": "rentId",
      "message": "Rental ID not found"
    }
  ]
}
```

4. 설명

Request 설명

1. **userPK** :
 - 취소 요청을 보낸 사용자 고유식별 ID.
 - 요청자가 해당 대여를 취소할 권한이 있는지 확인해야 함.
2. **rentId** :
 - 취소할 대여 ID.
 - 존재하지 않는 대여 ID의 경우 실패 처리.
3. **reason** :
 - 선택적으로 취소 사유를 제공할 수 있음.

Response 설명

1. **resultCode** :
 - "SUCCESS" : 취소 성공
 - "FAILURE" : 취소 실패
2. **message** :
 - 요청 성공/실패에 대한 설명 메시지
3. **data** :
 - **rentId** : 취소된 대여의 ID
 - **status** : 최종 대여 상태 (**canceled**)
 - **refundAmount** : 환불 금액 (환불이 있을 경우)
4. **errors** :
 - 취소 실패 시 오류 발생 필드와 메시지 반환

5. 예시

✅ 성공 요청/응답

Request

```
{
  "userPK": 001,
  "rentId": 101,
  "reason": "Change of plans"
}
```

Response

```
{
  "resultCode": "SUCCESS",
  "message": "Rental cancellation successful",
  "data": {
    "rentId": 101,
    "status": "canceled",
    "refundAmount": 1500
  }
}
```

❌ 실패 요청/응답 (**rentId** 가 존재하지 않음)

Request

```
{
  "userPK": 001,
  "rentId": 999, // 존재하지 않는 대여 ID
  "reason": "Change of plans"
}
```

Response

```
{
  "resultCode": "FAILURE",
  "message": "Rental cancellation failed",
  "errors": [
    {
      "field": "rentId",
      "message": "Rental ID not found"
    }
  ]
}
```

❌ 실패 요청/응답 (이미 완료된 대여 취소 불가)

Request

```
{
  "userPK": 001,
  "rentId": 102,
```

```
"reason": "No longer needed"
}
```

Response

```
{
  "resultCode": "FAILURE",
  "message": "Rental cancellation failed",
  "errors": [
    {
      "field": "rentId",
      "message": "Rental has already been completed and cannot be canceled"
    }
  ]
}
```

차량 정보 조회

≡ API	/user/rent/status
🕒 메소드	POST
≡ 보안	Bearer Token
☀ 상태	완료
≡ 설명	사용자가 특정 대여(<code>rentId</code>)와 연결된 차량의 실시간 상태 데이터를 조회합니다.

1. API 개요

- API 경로: `/user/rent/status`
- 메서드: `POST`
- 설명: 사용자가 특정 대여(`rentId`)와 연결된 차량의 실시간 상태 데이터를 조회합니다.
- 인증 필요: `Bearer Token`
- 기능:
 - 차량의 현재 위치(`location`), 목적지(`destination`), 주행 경로(`plannedPath`), 도착 여부(`isArrive`).
 - SLAM 기반 경로 및 맵 데이터 제공 (`SLAMMapData`).
 - 차량 상태 (`batteryLevel` , `lightBrightness` 등).
 - 장착된 모듈 및 옵션 상태 정보 포함.

2. Request

요청 형식

```
{
  "userPK": "Int",      // 사용자 고유식별 ID (필수)
  "rentId": 101         // 대여 고유 ID (필수)
}
```

요청 필드 설명

필드	타입	필수 여부	설명
<code>userPK</code>	<code>Int</code>	✓ 필수	데이터를 요청하는 사용자 고유 식별 ID
<code>rentId</code>	<code>Int</code>	✓ 필수	차량 상태를 조회할 대여 ID

3. Response

✓ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Real-time car data retrieved successfully",
  "data": {
    "isArrive": false, // 목적지 도착 여부
    "location": {
      "x": 12.313,     // 현재 차량 위치 - SLAM 좌표 (x)
      "y": 32.3232    // 현재 차량 위치 - SLAM 좌표 (y)
    }
  }
}
```

```

    },
    "destination": {
      "x": 40.1111, // 목적지 위치 - SLAM 좌표 (x)
      "y": 100.4194 // 목적지 위치 - SLAM 좌표 (y)
    },
    "ETA": "2025-01-13T14:30:00Z", // 예상 도착 시간
    "distanceTravelled": 120, // 주행 거리 (단위: km)
    "plannedPath": [ // SLAM 예상 경로 데이터
      {
        "x": 12.3200,
        "y": 32.3300
      },
      {
        "x": 15.4500,
        "y": 35.6000
      }
    ],
    "SLAMMapData": "base64-encoded-map-data", // SLAM 맵 데이터 (이미지 또는 맵 포맷)
    "status": { // 차량, 모듈 및 옵션 상태
      "carStatus": {
        "batteryLevel": 85, // 배터리 상태 (%)
        "lightBrightness": 80 // 조명 밝기 (%)
      },
      "moduleName": "캠핑카", // 장착된 모듈 이름
      "options": [ // 장착된 옵션 상태
        {
          "optionName": "물탱크",
          "optionStatus": "잔여량: 50L"
        },
        {
          "optionName": "배터리 팩",
          "optionStatus": "잔여량: 80%"
        }
      ]
    }
  }
}

```

❌ 실패 시

```

{
  "resultCode": "FAILURE",
  "message": "Failed to retrieve real-time car data",
  "errors": [
    {
      "field": "rentId",
      "message": "Invalid rentId"
    }
  ]
}

```

4. 설명

Request 설명

1. `userPK` :
 - 요청을 보낸 사용자 고유 식별 ID.
 - 권한 검증 및 데이터 필터링을 위해 사용.
2. `rentId` :
 - 해당 대여와 연결된 고유 식별자.
 - 차량 상태를 특정 대여와 연결하기 위해 필수.

Response 설명

1. `isArrive` :
 - 차량이 목적지에 도착했는지 여부 (`true` / `false`).
2. `location` :
 - 차량의 현재 **SLAM 좌표** (`x`, `y`).
3. `destination` :
 - 차량의 목적지 **SLAM 좌표** (`x`, `y`).
4. `ETA` :
 - 차량의 예상 도착 시간 (ISO 8601 형식).
5. `distanceTravelled` :
 - 차량이 현재까지 이동한 거리 (단위: km).
6. `plannedPath` :
 - **SLAM 데이터 기반 예상 경로.**
 - `x`, `y` 좌표 배열로 반환.
7. `SLAMMapData` :
 - 차량이 생성한 SLAM 맵의 데이터.
 - Base64로 인코딩된 이미지 또는 맵 포맷.
8. `status` :
 - `carStatus` :
 - 배터리 잔량, 조명 밝기 등 차량의 상태 정보.
 - `moduleName` :
 - 현재 장착된 모듈 이름.
 - `options` :
 - 현재 장착된 옵션과 상태.

5. 예시

✅ 성공 요청/응답

Request

```
{
  "userPK": 001,
```

```
"rentId": 101
}
```

Response

```
{
  "resultCode": "SUCCESS",
  "message": "Real-time car data retrieved successfully",
  "data": {
    "isArrive": false,
    "location": {
      "x": 12.313,
      "y": 32.3232
    },
    "destination": {
      "x": 40.1111,
      "y": 100.4194
    },
    "ETA": "2025-01-13T14:30:00Z",
    "distanceTravelled": 120,
    "plannedPath": [
      {
        "x": 12.3200,
        "y": 32.3300
      },
      {
        "x": 15.4500,
        "y": 35.6000
      }
    ],
    "SLAMMapData": "base64-encoded-map-data",
    "status": {
      "carStatus": {
        "batteryLevel": 85,
        "lightBrightness": 80
      },
      "moduleName": "캠핑카",
      "options": [
        {
          "optionName": "물탱크",
          "optionStatus": "잔여량: 50L"
        },
        {
          "optionName": "배터리 팩",
          "optionStatus": "잔여량: 80%"
        }
      ]
    }
  }
}
```

❌ 실패 요청/응답 (`rentId` 가 유효하지 않음)

Request

```
{
  "userPK": 001,
  "rentId": 999
}
```

Response

```
{
  "resultCode": "FAILURE",
  "message": "Failed to retrieve real-time car data",
  "errors": [
    {
      "field": "rentId",
      "message": "Invalid rentId"
    }
  ]
}
```

차량 대여 완료

≡ API	/user/rent/complete
🕒 메소드	POST
≡ 보안	Bearer Token
☀ 상태	완료
≡ 설명	사용자가 특정 대여(<code>rentId</code>)를 완료 처리합니다.

1. API 개요

- API 경로: /user/rent/complete
- 메서드: POST
- 설명: 사용자가 특정 대여(`rentId`)를 완료 처리합니다.
- 인증 필요: Bearer Token
- 기능:
 - 진행 중(`in-progress`) 상태의 대여만 완료 가능
 - `rent_status` 를 `completed` 로 변경
 - 총 주행 거리(`totalDistance`), 대여 기간(`totalDuration`), 최종 비용(`finalCost`) 계산 후 반환

2. Request

요청 형식

```
{
  "userPK": "Int",      // 사용자 고유식별 ID (필수)
  "rentId": 101         // 대여 고유 ID (필수)
}
```

요청 필드 설명

필드	타입	필수 여부	설명
<code>userPK</code>	<code>Int</code>	✓ 필수	대여를 완료하는 사용자 고유 식 ID
<code>rentId</code>	<code>Int</code>	✓ 필수	완료할 대여의 고유 ID

3. Response

✓ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Rental completed successfully",
  "data": {
    "rentId": 101,          // 완료된 대여 ID
    "totalDistance": 150,   // 총 주행 거리 (단위: km)
    "totalDuration": 3,     // 총 대여 기간 (단위: 일)
    "finalCost": 75000,     // 최종 대여 비용 (단위: 원)
    "status": "completed"  // 최종 대여 상태
  }
}
```



```
}  
}
```

❌ 실패 시

```
{  
  "resultCode": "FAILURE",  
  "message": "Failed to complete rental",  
  "errors": [  
    {  
      "field": "rentId",  
      "message": "Invalid rentId"  
    },  
    {  
      "field": "userId",  
      "message": "User does not have permission to complete this rental"  
    }  
  ]  
}
```

4. 설명

Request 설명

1. **userPK** :
 - 대여를 완료하려는 사용자.
 - 사용자 권한 확인 및 대여 기록을 연결하기 위해 필요.
2. **rentId** :
 - 대여와 연결된 고유 식별자.
 - 특정 대여를 식별하기 위해 필수.

Response 설명

1. 성공 (**SUCCESS**)
 - **resultCode** : "SUCCESS"
 - **message** : "Rental completed successfully"
 - **data** :
 - **rentId** : 완료된 대여의 ID.
 - **totalDistance** : 총 주행 거리 (단위: km).
 - **totalDuration** : 총 대여 기간 (단위: 일).
 - **finalCost** : 최종 대여 비용 (단위: 원).
 - **status** : "completed" (최종 상태).
2. 실패 (**FAILURE**)
 - **resultCode** : "FAILURE"
 - **message** : "Failed to complete rental"
 - **errors** : 요청 중 오류가 발생한 필드와 메시지 리스트.

5. 예시

✅ 성공 요청/응답

Request

```
{
  "userPK": 001,
  "rentId": 101
}
```

Response

```
{
  "resultCode": "SUCCESS",
  "message": "Rental completed successfully",
  "data": {
    "rentId": 101,
    "totalDistance": 150,
    "totalDuration": 3,
    "finalCost": 75000,
    "status": "completed"
  }
}
```

❌ 실패 요청/응답 (**rentId** 가 존재하지 않음)

Request

```
{
  "userId": "user123",
  "rentId": 999
}
```

Response

```
{
  "resultCode": "FAILURE",
  "message": "Failed to complete rental",
  "errors": [
    {
      "field": "rentId",
      "message": "Invalid rentId"
    }
  ]
}
```

❌ 실패 요청/응답 (이미 완료된 대여)

Request

```
{
  "userPK": 001,
```

```
"rentId": 101
}
```

Response

```
{
  "resultCode": "FAILURE",
  "message": "Failed to complete rental",
  "errors": [
    {
      "field": "rentId",
      "message": "Rental has already been completed"
    }
  ]
}
```

운행 중 목적지 설정

≡ API	/user/rent/setDestination
🕒 메소드	POST
≡ 보안	Bearer Token
☀ 상태	완료
≡ 설명	사용자가 운행 중인 차량의 목적지를 변경합니다.

1. API 개요

- API 경로: /user/rent/setDestination
- 메서드: POST
- 설명: 사용자가 운행 중인 차량의 목적지를 변경합니다.
- 인증 필요: Bearer Token
- 기능:
 - 진행 중 (in-progress) 상태의 대여만 목적지 변경 가능
 - SLAM 좌표 (x, y)를 목적지로 설정
 - 변경된 목적지를 DB에 반영

2. Request

요청 형식

```
{
  "userPK": 001, // 사용자 고유 식별 ID (필수)
  "rentId": 101, // 대여 고유 ID (필수)
  "destination": { // 새로운 목적지 (SLAM 좌표)
    "x": 12.7808,
    "y": 32.4151
  }
}
```

요청 필드 설명

필드	타입	필수 여부	설명
userPK	Int	✓ 필수	목적지를 설정하는 사용자 ID
rentId	Int	✓ 필수	목적지를 변경할 대여의 고유 ID
destination	Object	✓ 필수	새로운 목적지 (x, y 형식의 SLAM 좌표)
└ x	Float	✓ 필수	새로운 목적지 X 좌표
└ y	Float	✓ 필수	새로운 목적지 Y 좌표

3. Response

✓ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Destination set successfully",
  "data": {
    "rentId": 101,    // 대여 ID
    "destination": {
      "x": 12.7808,  // 설정된 목적지 X 좌표
      "y": 32.4151  // 설정된 목적지 Y 좌표
    },
    "status": "updated" // 목적지 업데이트 상태
  }
}
```

❌ 실패 시

```
{
  "resultCode": "FAILURE",
  "message": "Failed to set destination",
  "errors": [
    {
      "field": "rentId",
      "message": "Invalid rentId or rental is not in progress"
    }
  ]
}
```

4. 설명

Request 설명

1. **userPK** :
 - 목적지를 변경하려는 사용자 고유 식별 ID.
 - 사용자가 변경 권한이 있는지 확인.
2. **rentId** :
 - 대여와 연결된 고유 식별자.
 - 현재 진행 중인(**in-progress**) 대여만 변경 가능.
3. **destination** :
 - 변경할 **SLAM 좌표 (x, y)**.
 - 차량이 도착해야 할 새로운 목적지를 설정.

Response 설명

1. 성공 (**SUCCESS**)
 - **resultCode** : "SUCCESS"
 - **message** : "Destination set successfully"
 - **data** :
 - **rentId** : 설정된 대여의 ID.
 - **destination** : 변경된 목적지 좌표 (x, y).

- `status`: "updated" (목적지 업데이트 완료).

2. 실패 (`FAILURE`)

- `resultCode`: "FAILURE"
- `message`: "Failed to set destination"
- `errors`: 요청 중 오류가 발생한 필드와 메시지 리스트.

5. 예시

✅ 성공 요청/응답

Request

```
{ rentId": 101,
  "destination": {
    "x": 12.7808,
    "y": 32.4151
  }
}
```

Response

```
{
  "resultCode": "SUCCESS",
  "message": "Destination set successfully",
  "data": {
    "rentId": 101,
    "destination": {
      "x": 12.7808,
      "y": 32.4151
    },
    "status": "updated"
  }
}
```

❌ 실패 요청/응답 (`rentId` 가 존재하지 않음)

Request

```
{
  "userPK": 001,
  "rentId": 999,
  "destination": {
    "x": 12.7808,
    "y": 32.4151
  }
}
```

Response

```
{
  "resultCode": "FAILURE",
```

```
"message": "Failed to set destination",
"errors": [
  {
    "field": "rentId",
    "message": "Invalid rentId or rental is not in progress"
  }
]
}
```

챗봇 사용자 응답 생성

≡ API	/user/chatbot/query
🕒 메소드	POST
☀️ 상태	완료
≡ 설명	사용자가 자연어 질문을 입력하면 챗봇이 응답을 제공합니다.

1. API 개요

- API 경로: /user/chatbot/query
- 메서드: POST
- 설명: 사용자가 자연어 질문을 입력하면 챗봇이 응답을 제공합니다.
- 인증 필요: ❌ (userId 가 null 이면 로그인 없이도 사용 가능)
- 기능:
 - RAG 기반 문서 검색(참고 문서 반환 가능)
 - 개인 맞춤 추천 (로그인 시, 대여 이력/모듈 사용 내역 기반)
 - 자연어 질의 응답 지원

2. Request

요청 형식

```
{
  "userPK": null,          // 사용자 고유 식별 ID (선택, 로그인하지 않은 경우 null)
  "query": "String"       // 사용자가 입력한 질문 또는 요청 (필수)
}
```

요청 필드 설명

필드	타입	필수 여부	설명
userPK	Int , NULL	❌ 선택	로그인한 사용자의 ID, 로그인하지 않은 경우 null
query	String	✅ 필수	사용자의 질문 또는 요청

3. Response

✅ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Response generated successfully",
  "data": {
    "response": "캠핑을 계획 중이시군요! 추천드리는 모듈은 '캠핑 모듈'입니다. 사용하시면 더 편리한 캠핑을 즐기실 수 있습니다.",
    "sourceDocuments": [          // (선택) RAG에서 사용된 참고 문서
      {
        "title": "캠핑 모듈 정보",
        "content": "캠핑 모듈은 침대, 조리 공간, 태양광 충전 시스템을 포함합니다."
      }
    ]
  }
}
```



```
]
}
}
```

❌ 실패 시

```
{
  "resultCode": "FAILURE",
  "message": "Failed to generate response",
  "errors": [
    {
      "field": "query",
      "message": "Query cannot be empty"
    }
  ]
}
```

4. 설명

Request 설명

1. `userId` (선택)
 - 로그인한 사용자의 경우 **대여 이력, 모듈 사용 내역** 등을 참고하여 맞춤 응답 가능.
 - 로그인하지 않은 경우 `null` 허용.
2. `query` (필수)
 - 사용자가 입력하는 질문 또는 요청.
 - 예시:
 - "3명이 캠핑을 가려는데 어떤 모듈이 좋을까요?"
 - "50km 정도 이동할 수 있는 옵션을 추천해주세요."
 - "캠핑을 하면서 요리할 수 있는 기능이 필요해요."

Response 설명

1. 성공 (`SUCCESS`)
 - `resultCode` : `"SUCCESS"`
 - `message` : `"Response generated successfully"`
 - `data` :
 - `response` : 챗봇의 자연어 응답 메시지.
 - `sourceDocuments` (선택): RAG에서 사용된 참고 문서 리스트.
 - 문서 `title`, `content` 포함 가능.
2. 실패 (`FAILURE`)
 - `resultCode` : `"FAILURE"`
 - `message` : `"Failed to generate response"`
 - `errors` : 요청 중 오류가 발생한 필드와 메시지 리스트.

5. 예시

✅ 성공 요청/응답

Request

```
{
  "userId": "user123",
  "query": "3명이 캠핑을 가려는데 어떤 모듈이 좋을까요?"
}
```

Response

```
{
  "resultCode": "SUCCESS",
  "message": "Response generated successfully",
  "data": {
    "response": "캠핑을 계획 중이시군요! 추천드리는 모듈은 '캠핑 모듈'입니다. 사용하시면 더 편리한 캠핑을 즐기실 수 있습니다.",
    "sourceDocuments": [
      {
        "title": "캠핑 모듈 정보",
        "content": "캠핑 모듈은 침대, 조리 공간, 태양광 충전 시스템을 포함합니다."
      }
    ]
  }
}
```

❌ 실패 요청/응답 (빈 질문)

Request

```
{
  "userId": "user123",
  "query": ""
}
```

Response

```
{
  "resultCode": "FAILURE",
  "message": "Failed to generate response",
  "errors": [
    {
      "field": "query",
      "message": "Query cannot be empty"
    }
  ]
}
```

관리자 로그인

≡ API	/admin/login
🕒 메소드	POST
≡ 보안	bcrypt
☀ 상태	진행 중
≡ 설명	관리자가 로그인하면 JWT 토큰 을 발급받아 이후 API 요청에서 인증할 수 있습니다.

1. API 개요

- API 경로: /admin/login
- 메서드: POST
- 설명: 관리자가 로그인하면 **JWT 토큰**을 발급받아 이후 API 요청에서 인증할 수 있습니다.
- 기능:
 - 관리자 계정 인증
 - JWT 토큰 발급 및 만료 시간 포함
 - 관리자 역할(**role**) 반환

2. Request

요청 형식

```
{
  "adminId": "String", // 관리자 계정 ID (필수)
  "password": "String" // 관리자 계정 비밀번호 (필수)
}
```

요청 필드 설명

필드	타입	필수 여부	설명
adminId	String	✓ 필수	로그인할 관리자 계정 ID
password	String	✓ 필수	관리자 계정의 비밀번호

3. Response

✓ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Login successful",
  "data": {
    "token": "String", // 인증 토큰 (JWT)
    "adminId": "String", // 로그인한 관리자 계정 ID
    "role": "String", // 관리자 권한 (예: "master", "semi")
    "expiresAt": "2025-01-20T14:30:00Z" // 토큰 만료 시간 (ISO 8601 형식)
  }
}
```

❌ 실패 시

```
{
  "resultCode": "FAILURE",
  "message": "Login failed",
  "errors": [
    {
      "field": "adminId",
      "message": "Invalid admin ID or password"
    }
  ]
}
```

4. 설명

Request 설명

1. **adminId**:
 - 로그인 시 사용할 관리자 계정의 ID.
 - DB의 `admin.admin_id` 필드와 매칭.
2. **password**:
 - 해당 계정에 연결된 비밀번호.
 - 암호화된 상태로 저장되므로 로그인 시 해시 비교 수행.

Response 설명

1. 성공 (**SUCCESS**)
 - **resultCode**: "SUCCESS"
 - **message**: "Login successful"
 - **data**:
 - **token**: JWT 인증 토큰 (**Bearer Token** 으로 이후 요청에 포함).
 - **adminId**: 로그인된 관리자 계정 ID.
 - **role**: 관리자 권한 (**master** 또는 **semi**).
 - **expiresAt**: 토큰 만료 시간 (**ISO 8601** 형식).
2. 실패 (**FAILURE**)
 - **resultCode**: "FAILURE"
 - **message**: "Login failed"
 - **errors**: 오류가 발생한 필드와 메시지 반환.

5. 예시

✅ 성공 요청/응답

Request

```
{
  "adminId": "adminMaster",
```

```
"password": "securePassword123"
}
```

Response

```
{
  "resultCode": "SUCCESS",
  "message": "Login successful",
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "adminId": "adminMaster",
    "role": "master",
    "expiresAt": "2025-01-20T14:30:00Z"
  }
}
```

❌ 실패 요청/응답 (잘못된 비밀번호)

Request

```
{
  "adminId": "adminMaster",
  "password": "wrongPassword"
}
```

Response

```
{
  "resultCode": "FAILURE",
  "message": "Login failed",
  "errors": [
    {
      "field": "adminId",
      "message": "Invalid admin ID or password"
    }
  ]
}
```

❌ 실패 요청/응답 (존재하지 않는 계정)

Request

```
{
  "adminId": "nonExistentAdmin",
  "password": "somePassword"
}
```

Response

```
{
  "resultCode": "FAILURE",
  "message": "Login failed",
}
```

```
"errors": [  
  {  
    "field": "adminId",  
    "message": "Admin ID not found"  
  }  
]  
}
```

차량 관리 정보 조회

≡ API	/admin/vehicle/list
🕒 메소드	GET
≡ 보안	Bearer Token
☀ 상태	완료
≡ 설명	관리자가 등록된 차량 목록을 조회합니다.

1. API 개요

- API 경로: /admin/vehicle/list
- 메서드: GET
- 설명: 관리자가 등록된 차량 목록을 조회합니다.
- 인증 필요: ☒ Bearer Token 필요 (관리자 계정만 접근 가능)
- 기능:
 - 차량 상태(status) 필터링 가능 (active , inactive , maintenance)
 - 차량 번호(carNumber) 또는 VIN(vin) 검색 지원
 - 페이지네이션(page , pageSize) 지원
 - 마지막 정비(lastMaintenance), 마지막 사용(lastUsed) 정보 포함

2. Request

요청 형식

```
{
  "status": "String", // 차량 상태 필터 (선택, 예: "active", "inactive", "maintenance")
  "search": "String", // 특정 차량 번호 또는 VIN 검색 (선택)
  "page": 1, // 페이지 번호 (선택, 기본값: 1)
  "pageSize": 10 // 페이지 크기 (선택, 기본값: 10)
}
```

요청 필드 설명

필드	타입	필수 여부	설명
status	String	❌ 선택	차량 상태 필터링 (예: "active", "inactive", "maintenance")
search	String	❌ 선택	특정 차량 번호 또는 VIN 검색
page	Integer	❌ 선택	결과 페이지 번호 (기본값: 1)
pageSize	Integer	❌ 선택	한 페이지당 반환할 차량 개수 (기본값: 10)

3. Response

✅ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Vehicle data retrieved successfully",
}
```

```

"data": {
  "vehicles": [ // 차량 리스트
    {
      "vehicleId": 1,
      "vin": "ABC123456789XYZ",
      "carNumber": "PBV-1234",
      "status": "active",
      "currentRentId": "RENT001",
      "lastMaintenance": "2025-01-10",
      "lastUsed": "2024-12-31"
    },
    {
      "vehicleId": 2,
      "vin": "XYZ987654321ABC",
      "carNumber": "PBV-5678",
      "status": "inactive",
      "currentRentId": null,
      "lastMaintenance": "2024-12-20",
      "lastUsed": "2024-12-15"
    }
  ],
  "pagination": {
    "currentPage": 1,
    "totalPages": 5,
    "totalItems": 50,
    "pageSize": 10
  }
}

```

❌ 실패 시

```

{
  "resultCode": "FAILURE",
  "message": "Failed to retrieve vehicle data",
  "errors": [
    {
      "field": "status",
      "message": "Invalid status value"
    }
  ]
}

```

4. 설명

Request 설명

1. **status** :
 - 차량 상태 기준으로 필터링 (**active** , **inactive** , **maintenance**).
 - 기본적으로 모든 차량 조회.
2. **search** :
 - 차량 번호(**carNumber**) 또는 VIN(**vin**) 검색.

3. `page` :
 - 결과 페이지 번호 (`1` 부터 시작).
4. `pageSize` :
 - 한 페이지당 반환할 차량 개수 (기본값: `10`).

Response 설명

1. 성공 (`SUCCESS`)

- `resultCode` : `"SUCCESS"`
- `message` : `"Vehicle data retrieved successfully"`
- `data` :
 - `vehicles` : 차량 정보 리스트.
 - `pagination` : 페이지네이션 정보.

2. 실패 (`FAILURE`)

- `resultCode` : `"FAILURE"`
- `message` : `"Failed to retrieve vehicle data"`
- `errors` : 오류가 발생한 필드와 메시지 반환.

5. 예시

✅ 성공 요청/응답

Request

```
{
  "status": "active",
  "search": "PBV-1234",
  "page": 1,
  "pageSize": 5
}
```

Response

```
{
  "resultCode": "SUCCESS",
  "message": "Vehicle data retrieved successfully",
  "data": {
    "vehicles": [
      {
        "vehicleId": 1,
        "vin": "ABC123456789XYZ",
        "carNumber": "PBV-1234",
        "status": "active",
        "currentRentId": "RENT001",
        "lastMaintenance": "2025-01-10",
        "lastUsed": "2024-12-31"
      }
    ],
    "pagination": {
      "currentPage": 1,
```

```
    "totalPages": 1,  
    "totalItems": 1,  
    "pageSize": 5  
  }  
}  
}
```

❌ 실패 요청/응답 (잘못된 status 값)

Request

```
{  
  "status": "invalidStatus"  
}
```

Response

```
{  
  "resultCode": "FAILURE",  
  "message": "Failed to retrieve vehicle data",  
  "errors": [  
    {  
      "field": "status",  
      "message": "Invalid status value"  
    }  
  ]  
}
```

차량 관리 정보 등록

≡ API	/admin/vehicle/register
🕒 메소드	POST
≡ 보안	Bearer Token
☀️ 상태	완료
≡ 설명	관리자가 새로운 차량을 등록합니다.

1. API 개요

- API 경로: /admin/vehicle/register
- 메서드: POST
- 설명: 관리자가 새로운 차량을 등록합니다.
- 인증 필요: ☒ Bearer Token 필요 (관리자 계정만 접근 가능)
- 초기 상태: 차량이 등록될 때 기본적으로 inactive 상태로 저장됨.
- 기능:
 - 차량 차대번호(vin)와 차량 번호(carNumber) 중복 검사
 - 등록된 차량은 기본적으로 inactive 상태
 - 차량 ID(vehicle Id) 자동 생성

2. Request

요청 형식

```
{
  "vin": "String",          // 차량 차대번호 (필수)
  "carNumber": "String"     // 차량 번호판 (필수)
}
```

요청 필드 설명

필드	타입	필수 여부	설명
vin	String	☒ 필수	차량의 고유한 차대번호 (Vehicle Identification Number)
carNumber	String	☒ 필수	차량의 번호판 (예: "PBV-1234")

3. Response

☒ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Vehicle registered successfully",
  "data": {
    "vehicleId": 101,          // 시스템에서 생성된 차량 ID
    "vin": "ABC123456789XYZ", // 차량 차대번호
    "carNumber": "PBV-1234",  // 차량 번호판
    "status": "inactive",     // 등록된 차량의 초기 상태
  }
}
```

```

    "lastMaintenance": null, // 마지막 정비 날짜 (초기값 null)
    "lastUsed": null        // 마지막 사용 날짜 (초기값 null)
  }
}

```

❌ 실패 시

```

{
  "resultCode": "FAILURE",
  "message": "Failed to register vehicle",
  "errors": [
    {
      "field": "vin",
      "message": "VIN is already registered"
    },
    {
      "field": "carNumber",
      "message": "Car number is already registered"
    }
  ]
}

```

4. 설명

Request 설명

1. **vin** :
 - 필수 필드이며 **고유한 값**이어야 함.
 - 이미 존재하는 **vin** 이 있으면 **등록 불가**.
2. **carNumber** :
 - 필수 필드이며 **고유한 값**이어야 함.
 - 이미 존재하는 **carNumber** 가 있으면 **등록 불가**.

Response 설명

1. 성공 (**SUCCESS**)
 - resultCode** : "SUCCESS"
 - message** : "Vehicle registered successfully"
 - data** :
 - vehicleId** : 자동 생성된 차량의 고유 ID.
 - vin** : 등록된 차량의 차대번호.
 - carNumber** : 등록된 차량의 번호판.
 - status** : 기본적으로 **"inactive"** 상태.
 - lastMaintenance** : 마지막 정비 날짜 (처음 등록 시 **null**).
 - lastUsed** : 마지막 사용 날짜 (처음 등록 시 **null**).
2. 실패 (**FAILURE**)
 - resultCode** : "FAILURE"

- `message`: "Failed to register vehicle"
- `errors`: 요청 중 오류가 발생한 필드와 메시지 리스트.

5. 예시

✅ 성공 요청/응답

Request

```
{
  "vin": "ABC123456789XYZ",
  "carNumber": "PBV-1234"
}
```

Response

```
{
  "resultCode": "SUCCESS",
  "message": "Vehicle registered successfully",
  "data": {
    "vehicleId": 101,
    "vin": "ABC123456789XYZ",
    "carNumber": "PBV-1234",
    "status": "inactive",
    "lastMaintenance": null,
    "lastUsed": null
  }
}
```

❌ 실패 요청/응답 (이미 등록된 차량)

Request

```
{
  "vin": "ABC123456789XYZ",
  "carNumber": "PBV-1234"
}
```

Response

```
{
  "resultCode": "FAILURE",
  "message": "Failed to register vehicle",
  "errors": [
    {
      "field": "vin",
      "message": "VIN is already registered"
    },
    {
      "field": "carNumber",
      "message": "Car number is already registered"
    }
  ]
}
```

```
]
}
```

차량 관리 정보 수정

≡ API	/admin/vehicle/update/{vehicleId}
🕒 메소드	PUT
≡ 보안	Bearer Token
☀ 상태	완료
≡ 설명	관리자가 등록된 특정 차량 정보를 수정합니다.

1. API 개요

- API 경로: /admin/vehicle/update/{vehicleId}
- 메서드: PUT
- 설명: 관리자가 등록된 특정 차량 정보를 수정합니다.
- 인증 필요: ☒ Bearer Token 필요 (관리자 계정만 접근 가능)
- 기능:
 - 차량 번호(carNumber) 변경 가능
 - 차량 상태(status) 변경 가능 (active, inactive, maintenance)
 - 차량 차대번호(vin)는 변경 불가

2. Request

요청 형식

```
{
  "carNumber": "String", // 새로운 차량 번호 (선택)
  "status": "String"     // 차량 상태 (선택, 예: "active", "inactive", "maintenance")
}
```

요청 필드 설명

필드	타입	필수 여부	설명
carNumber	String	❌ 선택	새로운 차량 번호 (예: "PBV-5678")
status	String	❌ 선택	차량 상태 변경 ("active", "inactive", "maintenance")

3. Response

✅ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Vehicle updated successfully",
  "data": {
    "vehicleId": 101, // 차량 ID
    "vin": "ABC987654321XYZ", // 기존 차대번호 (수정 불가)
    "carNumber": "PBV-5678", // 업데이트된 차량 번호
    "status": "active" // 업데이트된 차량 상태
  }
}
```

```
}  
}
```

❌ 실패 시

```
{  
  "resultCode": "FAILURE",  
  "message": "Failed to update vehicle",  
  "errors": [  
    {  
      "field": "carNumber",  
      "message": "Car number is already registered"  
    },  
    {  
      "field": "status",  
      "message": "Invalid status value"  
    }  
  ]  
}
```

4. 설명

Request 설명

1. `carNumber` (선택)
 - 차량 번호 변경 시 사용.
 - 중복된 차량 번호(`carNumber`)는 허용되지 않음.
2. `status` (선택)
 - 차량 상태를 변경할 때 사용.
 - `"active"`, `"inactive"`, `"maintenance"` 중 하나만 허용됨.

Response 설명

1. 성공 (`SUCCESS`)
 - `resultCode`: `"SUCCESS"`
 - `message`: `"Vehicle updated successfully"`
 - `data`:
 - `vehicleId`: 업데이트된 차량의 ID.
 - `vin`: 기존 차량 차대번호 (수정 불가).
 - `carNumber`: 변경된 차량 번호.
 - `status`: 변경된 차량 상태.
2. 실패 (`FAILURE`)
 - `resultCode`: `"FAILURE"`
 - `message`: `"Failed to update vehicle"`
 - `errors`: 오류가 발생한 필드와 메시지 리스트.

5. 예시

✅ 성공 요청/응답

Request

```
{
  "carNumber": "PBV-5678",
  "status": "active"
}
```

Response

```
{
  "resultCode": "SUCCESS",
  "message": "Vehicle updated successfully",
  "data": {
    "vehicleId": 101,
    "vin": "ABC987654321XYZ",
    "carNumber": "PBV-5678",
    "status": "active"
  }
}
```

❌ 실패 요청/응답 (이미 등록된 차량 번호 사용)

Request

```
{
  "carNumber": "PBV-1234"    // 이미 등록된 차량 번호
}
```

Response

```
{
  "resultCode": "FAILURE",
  "message": "Failed to update vehicle",
  "errors": [
    {
      "field": "carNumber",
      "message": "Car number is already registered"
    }
  ]
}
```

❌ 실패 요청/응답 (잘못된 상태 값)

Request

```
{
  "status": "invalidStatus"
}
```

Response

```
{
  "resultCode": "FAILURE",
  "message": "Failed to update vehicle",
  "errors": [
    {
      "field": "status",
      "message": "Invalid status value"
    }
  ]
}
```

차량 관리 정보 삭제

≡ API	/admin/vehicle/delete/{vehicleId}
🕒 메소드	DELETE
≡ 보안	Bearer Token
⚙ 상태	완료
≡ 설명	관리자가 등록된 특정 차량을 삭제합니다.

1. API 개요

- API 경로: /admin/vehicle/delete/{vehicleId}
- 메서드: DELETE
- 설명: 관리자가 등록된 특정 차량을 삭제합니다.
- 인증 필요:  Bearer Token 필요 (관리자 계정만 접근 가능)
- 주의사항:
 - 진행 중(in-progress)인 대여가 있는 차량은 삭제 불가
 - 삭제 전, 차량이 대여 이력(rent_history)에 존재하는지 확인 필요

2. Request

요청 형식

- URL 경로 파라미터:
 - {vehicleId}: 삭제할 차량의 고유 ID (필수)
- 헤더:

Authorization: Bearer <관리자_토큰>

3. Response

✅ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Vehicle deleted successfully",
  "data": {
    "vehicleId": 101,           // 삭제된 차량 ID
    "vin": "ABC987654321XYZ", // 삭제된 차량의 차대번호
    "carNumber": "PBV-1234"   // 삭제된 차량 번호
  }
}
```

❌ 실패 시

🚫 인증 실패 (401 Unauthorized)

```
{
  "resultCode": "FAILURE",
  "message": "Unauthorized access",
  "errors": [
    {
      "field": "Authorization",
      "message": "Invalid or missing Bearer Token"
    }
  ]
}
```

삭제 실패 (404 Not Found) - 존재하지 않는 차량

```
{
  "resultCode": "FAILURE",
  "message": "Failed to delete vehicle",
  "errors": [
    {
      "field": "vehicleId",
      "message": "Vehicle not found or already deleted"
    }
  ]
}
```

삭제 실패 (400 Bad Request) - 차량이 대여 중

```
{
  "resultCode": "FAILURE",
  "message": "Vehicle cannot be deleted while it is in use",
  "errors": [
    {
      "field": "vehicleId",
      "message": "Vehicle is currently rented and cannot be deleted"
    }
  ]
}
```

4. 설명

Request 설명

1. URL 경로 파라미터:

- `{vehicleId}` : 삭제하려는 차량의 고유 ID.

2. Authorization 헤더:

- `Bearer Token` 을 포함하여 관리자 인증을 수행.

Response 설명

1. 성공 (SUCCESS)

- `resultCode` : "SUCCESS"

- `message` : "Vehicle deleted successfully"
- `data` :
 - `vehicleId` : 삭제된 차량의 ID.
 - `vin` : 삭제된 차량의 차대번호.
 - `carNumber` : 삭제된 차량의 번호.

2. 실패 (`FAILURE`)

- `resultCode` : `"FAILURE"`
- `message` : `"Failed to delete vehicle"`
- `errors` :
 - 차량이 존재하지 않거나 이미 삭제된 경우.
 - 차량이 현재 대여 중(`in-progress`)이라 삭제 불가능한 경우.
 - 관리자 인증 실패(`Unauthorized`).

5. 예시

✅ 성공 요청/응답

Request

- `DELETE` `/admin/vehicle/delete/101`
- 헤더:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Response

```
{
  "resultCode": "SUCCESS",
  "message": "Vehicle deleted successfully",
  "data": {
    "vehicleId": 101,
    "vin": "ABC987654321XYZ",
    "carNumber": "PBV-1234"
  }
}
```

❌ 실패 요청/응답 (`차량이 존재하지 않음`)

Request

- `DELETE` `/admin/vehicle/delete/999`
- 헤더:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Response

```
{
  "resultCode": "FAILURE",
```

```
"message": "Failed to delete vehicle",
"errors": [
  {
    "field": "vehicleId",
    "message": "Vehicle not found or already deleted"
  }
]
```

❌ 실패 요청/응답 (차량이 현재 대여 중)

Request

- DELETE `/admin/vehicle/delete/102`
- 헤더:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Response

```
{
  "resultCode": "FAILURE",
  "message": "Vehicle cannot be deleted while it is in use",
  "errors": [
    {
      "field": "vehicleId",
      "message": "Vehicle is currently rented and cannot be deleted"
    }
  ]
}
```

❌ 실패 요청/응답 (잘못된 인증 토큰)

Request

- DELETE `/admin/vehicle/delete/101`
- 헤더 (잘못된 토큰 또는 없음):

```
Authorization: Bearer invalid_token
```

Response

```
{
  "resultCode": "FAILURE",
  "message": "Unauthorized access",
  "errors": [
    {
      "field": "Authorization",
      "message": "Invalid or missing Bearer Token"
    }
  ]
}
```

모듈 세트 정보 조회

≡ API	/admin/module-set/list
🕒 메소드	GET
≡ 보안	Bearer Token
☀ 상태	완료
≡ 설명	관리자가 등록된 모듈 세트 목록을 조회합니다.

1. API 개요

- API 경로: /admin/module-set/list
- 메서드: GET
- 설명: 관리자가 등록된 모듈 세트 목록을 조회합니다.
- 인증 필요: ☒ Bearer Token 필요 (관리자 계정만 접근 가능)
- 기능:
 - 모듈 세트 이름(moduleSetName) 검색 가능
 - 페이지네이션(page , pageSize) 지원
 - 모듈 세트별 포함된 옵션 목록 제공

2. Request

요청 형식

```
{
  "search": "String",      // 모듈 세트 이름 검색 키워드 (선택)
  "page": 1,               // 페이지 번호 (선택, 기본값: 1)
  "pageSize": 10           // 페이지 크기 (선택, 기본값: 10)
}
```

요청 필드 설명

필드	타입	필수 여부	설명
search	String	✗ 선택	모듈 세트 이름 검색 키워드
page	Integer	✗ 선택	결과 페이지 번호 (기본값: 1)
pageSize	Integer	✗ 선택	한 페이지당 반환할 세트 개수 (기본값: 10)

3. Response

✅ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Module sets retrieved successfully",
  "data": {
    "moduleSets": [
      {
        "moduleSetId": 101,           // 모듈 세트 ID

```

```

    "moduleSetName": "캠핑카 모듈 세트", // 모듈 세트 이름
    "totalCost": 2500.00, // 총 가격
    "imgUrls": [ // 이미지 URL 목록
        "https://example.com/images/module-set-101.jpg"
    ],
    "description": "캠핑을 위한 완벽한 모듈 세트",
    "options": [ // 세트에 포함된 옵션 리스트
        {
            "optionId": 201,
            "optionName": "배터리 팩",
            "quantity": 1
        },
        {
            "optionId": 202,
            "optionName": "태양광 패널",
            "quantity": 2
        }
    ]
},
{
    "pagination": { // 페이지네이션 정보
        "currentPage": 1,
        "totalPages": 3,
        "totalItems": 25,
        "pageSize": 10
    }
}
}
}

```

❌ 실패 시

```

{
  "resultCode": "FAILURE",
  "message": "Failed to retrieve module sets",
  "errors": [
    {
      "field": "search",
      "message": "Invalid search value"
    }
  ]
}

```

4. 설명

Request 설명

1. `search` (선택)
 - 특정 모듈 세트 이름을 검색할 때 사용.
 - 입력하지 않으면 모든 모듈 세트를 반환.
2. `page` (선택)
 - 결과 페이지 번호 (1 부터 시작).
3. `pageSize` (선택)

- 한 페이지에 반환될 세트 개수를 설정 (기본값: 10).

Response 설명

1. 성공 (SUCCESS)

- `resultCode` : "SUCCESS"
- `message` : "Module sets retrieved successfully"
- `data` :
 - `moduleSets` : 모듈 세트 목록
 - `moduleSetId` : 모듈 세트의 고유 ID.
 - `moduleSetName` : 모듈 세트 이름.
 - `totalCost` : 총 가격.
 - `imgUrls` : 이미지 URL 목록.
 - `description` : 모듈 세트 설명.
 - `options` : 포함된 옵션 목록
 - `optionId` : 옵션 ID.
 - `optionName` : 옵션 이름.
 - `quantity` : 해당 세트에 포함된 개수.
 - `pagination` : 페이지네이션 정보.

2. 실패 (FAILURE)

- `resultCode` : "FAILURE"
- `message` : "Failed to retrieve module sets"
- `errors` : 오류가 발생한 필드와 메시지 리스트.

5. 예시

✅ 성공 요청/응답

Request

```
{
  "search": "캠핑",
  "page": 1,
  "pageSize": 5
}
```

Response

```
{
  "resultCode": "SUCCESS",
  "message": "Module sets retrieved successfully",
  "data": {
    "moduleSets": [
      {
        "moduleSetId": 101,
        "moduleSetName": "캠핑카 모듈 세트",
        "totalCost": 2500.00,
```

```

    "imgUrls": [
      "https://example.com/images/module-set-101.jpg"
    ],
    "description": "캠핑을 위한 완벽한 모듈 세트",
    "options": [
      {
        "optionId": 201,
        "optionName": "배터리 팩",
        "quantity": 1
      },
      {
        "optionId": 202,
        "optionName": "태양광 패널",
        "quantity": 2
      }
    ]
  }
},
"pagination": {
  "currentPage": 1,
  "totalPages": 1,
  "totalItems": 1,
  "pageSize": 5
}
}
}

```

❌ 실패 요청/응답 (잘못된 검색어 입력)

Request

```

{
  "search": "비행"
}

```

Response

```

{
  "resultCode": "FAILURE",
  "message": "Failed to retrieve module sets",
  "errors": [
    {
      "field": "search",
      "message": "Invalid search value"
    }
  ]
}

```

모듈 세트 정보 삭제

≡ API	/admin/module-set/delete/{moduleSetId}
🕒 메소드	DELETE
≡ 보안	Bearer Token
☀ 상태	완료
≡ 설명	관리자가 기존에 등록된 모듈 세트를 삭제합니다.

1. API 개요

- API 경로: /admin/module-set/delete/{moduleSetId}
- 메서드: DELETE
- 설명: 관리자가 기존에 등록된 모듈 세트를 삭제합니다.
- 인증 필요: ☒ Bearer Token 필요 (관리자 계정만 접근 가능)
- 제한 사항:
 - 현재 사용 중인 모듈 세트는 삭제할 수 없음
 - 존재하지 않는 모듈 세트 삭제 시 오류 반환

2. Request

요청 형식

- DELETE 메서드는 요청 본문이 필요하지 않으며, 삭제할 모듈 세트의 ID를 URL 경로에서 전달합니다.

요청 예시

- URL: /admin/module-set/delete/SET001

3. Response

✅ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Module set deleted successfully",
  "data": {
    "moduleSetId": 101,           // 삭제된 모듈 세트 ID
    "moduleSetName": "캠핑카 모듈 세트", // 삭제된 모듈 세트 이름
    "totalCost": 2500.00,        // 삭제된 모듈 세트 총 가격
    "imgUrls": [                 // 삭제된 이미지 URL 목록
      "https://example.com/images/module-set-101.jpg"
    ],
    "description": "캠핑에 적합한 모듈 세트",
    "options": [                 // 삭제된 세트의 옵션 목록
      {
        "optionId": 201,
        "optionName": "배터리 팩",
        "quantity": 1
      },
    ],
  },
}
```

```
{
  "optionId": 202,
  "optionName": "물탱크",
  "quantity": 2
}
]
```

❌ 실패 시

존재하지 않는 세트 삭제 시

```
{
  "resultCode": "FAILURE",
  "message": "Failed to delete module set",
  "errors": [
    {
      "field": "moduleSetId",
      "message": "Module set not found"
    }
  ]
}
```

현재 사용 중인 세트 삭제 시

```
{
  "resultCode": "FAILURE",
  "message": "Failed to delete module set",
  "errors": [
    {
      "field": "moduleSetId",
      "message": "Module set is currently in use and cannot be deleted"
    }
  ]
}
```

4. 설명

Request 설명

1. URL 경로 파라미터:

- `{moduleSetId}`: 삭제할 모듈 세트의 **고유 ID**를 URL에서 전달.
- 예: `/admin/module-set/delete/101`

2. 인증 필요:

- 요청 시 ****관리자 Bearer Token** *이 필요.
- 잘못된 토큰이 제공되면 **401 Unauthorized** 응답 반환.

Response 설명

1. 성공 (**SUCCESS**)

- `resultCode`: "SUCCESS"
- `message`: "Module set deleted successfully"
- `data`:
 - `moduleSetId`: 삭제된 모듈 세트 ID.
 - `moduleSetName`: 삭제된 모듈 세트 이름.
 - `totalCost`: 삭제된 총 가격.
 - `imgUrls`: 삭제된 이미지 URL 목록.
 - `description`: 삭제된 설명.
 - `options`: 삭제된 옵션 목록.

2. 실패 (FAILURE)

- `resultCode`: "FAILURE"
- `message`: "Failed to delete module set"
- `errors`: 요청 중 오류가 발생한 필드와 메시지 리스트.

5. 예시

✅ 성공 요청/응답

모듈 세트 삭제 성공

• Request

- `DELETE` /admin/module-set/delete/101

• Response

```
{
  "resultCode": "SUCCESS",
  "message": "Module set deleted successfully",
  "data": {
    "moduleSetId": 101,
    "moduleSetName": "캠핑카 모듈 세트",
    "totalCost": 2500.00,
    "imgUrls": [
      "https://example.com/images/module-set-101.jpg"
    ],
    "description": "캠핑에 적합한 모듈 세트",
    "options": [
      {
        "optionId": 201,
        "optionName": "배터리 팩",
        "quantity": 1
      },
      {
        "optionId": 202,
        "optionName": "물탱크",
        "quantity": 2
      }
    ]
  }
}
```

❌ 실패 요청/응답

존재하지 않는 모듈 세트 삭제 요청

- Request

- `DELETE /admin/module-set/delete/9999`

- Response

```
{
  "resultCode": "FAILURE",
  "message": "Failed to delete module set",
  "errors": [
    {
      "field": "moduleSetId",
      "message": "Module set not found"
    }
  ]
}
```

현재 사용 중인 모듈 세트 삭제 요청

- Request

- `DELETE /admin/module-set/delete/102` (현재 사용 중인 세트)

- Response

```
{
  "resultCode": "FAILURE",
  "message": "Failed to delete module set",
  "errors": [
    {
      "field": "moduleSetId",
      "message": "Module set is currently in use and cannot be deleted"
    }
  ]
}
```

모듈 관리 정보 조회

≡ API	/admin/module/list
🕒 메소드	GET
≡ 보안	Bearer Token
☀ 상태	완료
≡ 설명	관리자가 시스템에 등록된 모듈 목록을 조회합니다.

1. API 개요

- API 경로: /admin/module/list
- 메서드: GET
- 설명: 관리자가 시스템에 등록된 모듈 목록을 조회합니다.
- 인증 필요: ☒ Bearer Token 필요 (관리자 계정만 접근 가능)
- 기능:
 - 모듈 이름 및 NFC 태그 검색
 - 모듈 상태별 필터링 (active, inactive, maintenance)
 - 페이지네이션 지원
 - 현재 대여 여부 및 정비 기록 포함

2. Request

요청 형식

```
{
  "search": "String",      // 모듈 이름 또는 NFC 태그 검색 키워드 (선택)
  "status": "String",      // 모듈 상태 필터 (선택, 예: "active", "inactive", "maintenanc
e")
  "page": 1,               // 페이지 번호 (선택, 기본값: 1)
  "pageSize": 10           // 페이지 크기 (선택, 기본값: 10)
}
```

요청 필드 설명

필드	타입	필수 여부	설명
search	String	✗ 선택	모듈 이름 또는 NFC 태그 검색 키워드
status	String	✗ 선택	모듈 상태 필터링 (active, inactive, maintenance)
page	Integer	✗ 선택	결과 페이지 번호 (기본값: 1)
pageSize	Integer	✗ 선택	한 페이지당 반환할 모듈 개수 (기본값: 10)

3. Response

✓ 성공 시

```
{
  "resultCode": "SUCCESS",
```

```

"message": "Module list retrieved successfully",
"data": {
  "modules": [                                // 모듈 리스트
    {
      "moduleId": 101,                        // 모듈 ID
      "moduleNfcTagId": "NFC123456", // NFC 태그 ID
      "moduleType": "배터리 모듈", // 모듈 유형
      "moduleSize": "5x5x5",           // 모듈 크기
      "moduleCost": 1500.00,           // 모듈 비용
      "status": "active",               // 모듈 상태 (active/inactive/maintenance)
      "currentRentId": 1001,           // 현재 대여 ID (없으면 null)
      "lastMaintenance": "2025-01-01", // 마지막 정비 날짜
      "lastUsed": "2025-01-10"        // 마지막 사용 날짜
    },
    {
      "moduleId": 102,
      "moduleNfcTagId": "NFC654321",
      "moduleType": "태양광 패널",
      "moduleSize": "10x10x10",
      "moduleCost": 2000.00,
      "status": "inactive",
      "currentRentId": null,
      "lastMaintenance": "2024-12-15",
      "lastUsed": null
    }
  ],
  "pagination": {                            // 페이지네이션 정보
    "currentPage": 1,
    "totalPages": 3,
    "totalItems": 25,
    "pageSize": 10
  }
}

```

❌ 실패 시

```

{
  "resultCode": "FAILURE",
  "message": "Failed to retrieve module list",
  "errors": [
    {
      "field": "status",
      "message": "Invalid status value"
    }
  ]
}

```

4. 설명

Request 설명

1. **search**:
 - 모듈 이름이나 NFC 태그를 검색 가능.

- 입력하지 않으면 모든 모듈을 반환.
2. `status` :
 - 특정 상태의 모듈만 필터링 (`active` , `inactive` , `maintenance`).
 3. `page` :
 - 결과 페이지 번호를 지정.
 4. `pageSize` :
 - 한 페이지당 반환할 모듈 개수를 설정.

Response 설명

1. 성공 (`SUCCESS`)

- `resultCode` : "SUCCESS"
- `message` : "Module list retrieved successfully"
- `data` :
 - `modules` : 조회된 모듈 목록.
 - `moduleId` : 모듈 ID.
 - `moduleNfcTagId` : NFC 태그 ID.
 - `moduleType` : 모듈 유형.
 - `moduleSize` : 모듈 크기.
 - `moduleCost` : 모듈 가격.
 - `status` : 모듈 상태 (`active` , `inactive` , `maintenance`).
 - `currentRentId` : 현재 대여 ID (`null` 일 경우 대여 중이 아님).
 - `lastMaintenance` : 마지막 정비 날짜.
 - `lastUsed` : 마지막 사용 날짜.
 - `pagination` : 페이지네이션 정보 (`currentPage` , `totalPages` , `totalItems` , `pageSize` 포함).

2. 실패 (`FAILURE`)

- `resultCode` : "FAILURE"
- `message` : "Failed to retrieve module list"
- `errors` : 요청 중 오류가 발생한 필드와 메시지 리스트.

5. 예시

✅ 성공 요청/응답

NFC 태그로 모듈 검색

```
{
  "search": "NFC123",
  "status": "active",
  "page": 1,
  "pageSize": 5
}
```

```
{
  "resultCode": "SUCCESS",
```

```

"message": "Module list retrieved successfully",
"data": {
  "modules": [
    {
      "moduleId": 101,
      "moduleNfcTagId": "NFC123456",
      "moduleType": "배터리 모듈",
      "moduleSize": "5x5x5",
      "moduleCost": 1500.00,
      "status": "active",
      "currentRentId": 1001,
      "lastMaintenance": "2025-01-01",
      "lastUsed": "2025-01-10"
    }
  ],
  "pagination": {
    "currentPage": 1,
    "totalPages": 1,
    "totalItems": 1,
    "pageSize": 5
  }
}

```

❌ 실패 요청/응답

잘못된 상태값 필터링

```

{
  "status": "invalidStatus"
}

```

```

{
  "resultCode": "FAILURE",
  "message": "Failed to retrieve module list",
  "errors": [
    {
      "field": "status",
      "message": "Invalid status value"
    }
  ]
}

```

모듈 관리 정보 등록

≡ API	/admin/module/register
🕒 메소드	POST
≡ 보안	Bearer Token
☀️ 상태	완료
≡ 설명	관리자가 새로운 모듈 정보를 시스템에 등록합니다.

1. API 개요

- API 경로: /admin/module/register
- 메서드: POST
- 설명: 관리자가 새로운 모듈 정보를 시스템에 등록합니다.
 - 각 모듈은 NFC 태그를 통해 식별됩니다.
 - 등록된 모듈은 기본적으로 inactive 상태로 설정됩니다.
- 인증 필요: ☒ Bearer Token 필요 (관리자 계정만 접근 가능)

2. Request

요청 형식

```
{
  "moduleNfcTagId": "NFC123456", // NFC 태그 ID (고유 값)
  "moduleType": "배터리 모듈", // 모듈 유형 (예: 배터리, 태양광 패널 등)
  "moduleSize": "5x5x5", // 모듈 크기
  "moduleCost": 1500.00 // 모듈 비용
}
```

요청 필드 설명

필드	타입	필수 여부	설명
moduleNfcTagId	String	☒ 필수	NFC 태그 ID (유일해야 함)
moduleType	String	☒ 필수	모듈 유형 (예: "배터리 모듈", "태양광 패널")
moduleSize	String	☒ 필수	모듈 크기 (예: "5x5x5")
moduleCost	Decimal(10,2)	☒ 필수	모듈 비용 (소수점 포함 가능)

3. Response

☒ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Module registered successfully",
  "data": {
    "moduleId": 101, // 등록된 모듈 ID
    "moduleNfcTagId": "NFC123456", // NFC 태그 ID
    "moduleType": "배터리 모듈", // 모듈 유형
    "moduleSize": "5x5x5", // 모듈 크기
  }
}
```

```

    "moduleCost": 1500.00,          // 모듈 비용
    "status": "inactive"           // 초기 모듈 상태 (기본값: inactive)
  }
}

```

❌ 실패 시

```

{
  "resultCode": "FAILURE",
  "message": "Failed to register module",
  "errors": [
    {
      "field": "moduleNfcTagId",
      "message": "NFC tag ID is already in use"
    },
    {
      "field": "moduleType",
      "message": "Invalid module type"
    }
  ]
}

```

4. 설명

Request 설명

1. `moduleNfcTagId` :
 - 모듈과 연결된 NFC 태그의 **고유 ID**.
 - 이미 등록된 `moduleNfcTagId` 는 허용되지 않음.
2. `moduleType` :
 - 모듈의 유형을 나타냄 (예: "배터리 모듈", "태양광 패널" 등).
3. `moduleSize` :
 - 모듈 크기를 나타냄.
 - 예: "5x5x5" 형식.
4. `moduleCost` :
 - 모듈의 가격을 나타냄.
 - 소수점 포함 가능 (`DECIMAL(10,2)`) 형식.

Response 설명

1. 성공 (`SUCCESS`)
 - `resultCode` : "SUCCESS"
 - `message` : "Module registered successfully"
 - `data` :
 - `moduleId` : 시스템에서 자동 생성한 모듈의 고유 ID.
 - `moduleNfcTagId` : 등록된 NFC 태그 ID.
 - `moduleType` : 등록된 모듈 유형.

- `moduleSize`: 등록된 모듈 크기.
- `moduleCost`: 등록된 모듈 비용.
- `status`: `"inactive"` (기본 상태).

2. 실패 (`FAILURE`)

- `resultCode`: `"FAILURE"`
- `message`: `"Failed to register module"`
- `errors`: 오류 발생 필드와 메시지 포함.

5. 예시

✅ 성공 요청/응답

Request

```
{
  "moduleNfcTagId": "NFC123456",
  "moduleType": "배터리 모듈",
  "moduleSize": "5x5x5",
  "moduleCost": 1500.00
}
```

Response

```
{
  "resultCode": "SUCCESS",
  "message": "Module registered successfully",
  "data": {
    "moduleId": 101,
    "moduleNfcTagId": "NFC123456",
    "moduleType": "배터리 모듈",
    "moduleSize": "5x5x5",
    "moduleCost": 1500.00,
    "status": "inactive"
  }
}
```

❌ 실패 요청/응답

Request (중복된 NFC 태그 ID)

```
{
  "moduleNfcTagId": "NFC123456",
  "moduleType": "배터리 모듈",
  "moduleSize": "5x5x5",
  "moduleCost": 1500.00
}
```

Response

```
{
  "resultCode": "FAILURE",
```

```
"message": "Failed to register module",
"errors": [
  {
    "field": "moduleNfcTagId",
    "message": "NFC tag ID is already in use"
  }
]
```

❌ 실패 요청/응답

Request (잘못된 모듈 유형)

```
{
  "moduleNfcTagId": "NFC789123",
  "moduleType": "잘못된 유형",
  "moduleSize": "5x5x5",
  "moduleCost": 1500.00
}
```

Response

```
{
  "resultCode": "FAILURE",
  "message": "Failed to register module",
  "errors": [
    {
      "field": "moduleType",
      "message": "Invalid module type"
    }
  ]
}
```

모듈 관리 정보 수정

≡ API	/admin/module/update/{moduleId}
🕒 메소드	PUT
≡ 보안	Bearer Token
☀ 상태	완료
≡ 설명	관리자가 기존에 등록된 모듈 정보를 수정합니다.

1. API 개요

- API 경로: /admin/module/update/{moduleId}
- 메서드: PUT
- 설명: 관리자가 기존에 등록된 모듈 정보를 수정합니다.
 - NFC 태그 ID, 크기, 비용 등의 정보를 업데이트할 수 있습니다.
 - 다른 모듈에서 사용 중인 NFC 태그로 변경할 수 없습니다.
- 인증 필요: ☒ Bearer Token 필요 (관리자 계정만 접근 가능)

2. Request

요청 경로

/admin/module/update/{moduleId}

- moduleId → 수정할 모듈의 ID

요청 형식

```
{
  "moduleNfcTagId": "NFC987654", // NFC 태그 ID (선택, 중복 불가)
  "moduleType": "태양광 패널", // 모듈 유형 (선택)
  "moduleSize": "10x10x10", // 모듈 크기 (선택)
  "moduleCost": 2500.00 // 모듈 비용 (선택, 양수)
}
```

요청 필드 설명

필드	타입	필수 여부	설명
moduleNfcTagId	String	선택	NFC 태그 ID (중복 불가)
moduleType	String	선택	모듈 유형 (예: "배터리 모듈", "태양광 패널")
moduleSize	String	선택	모듈 크기 (예: "10x10x10")
moduleCost	Decimal(10,2)	선택	모듈 비용 (소수점 포함 가능, 양수)

3. Response

✅ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Module updated successfully",
}
```

```

"data": {
  "moduleId": 101,           // 수정된 모듈 ID
  "moduleNfcTagId": "NFC987654", // 수정된 NFC 태그 ID
  "moduleType": "태양광 패널", // 수정된 모듈 유형
  "moduleSize": "10x10x10",   // 수정된 모듈 크기
  "moduleCost": 2500.00      // 수정된 모듈 비용
}
}

```

❌ 실패 시

```

{
  "resultCode": "FAILURE",
  "message": "Failed to update module",
  "errors": [
    {
      "field": "moduleNfcTagId",
      "message": "NFC tag ID is already in use"
    },
    {
      "field": "moduleCost",
      "message": "Cost must be a positive number"
    }
  ]
}

```

4. 설명

Request 설명

1. `moduleId` :
 - `{moduleId}` 경로에 포함된 필드로, 수정하려는 **모듈의 고유 ID**.
2. `moduleNfcTagId` :
 - NFC 태그 ID를 업데이트할 수 있습니다.
 - 다른 모듈에서 사용 중인 ID로 변경할 수 없습니다.
3. `moduleType` :
 - 모듈의 유형을 변경할 수 있습니다. (예: "배터리 모듈", "태양광 패널")
4. `moduleSize` :
 - 모듈의 크기를 변경할 수 있습니다. (예: "10x10x10")
5. `moduleCost` :
 - 모듈 비용을 업데이트할 수 있습니다.
 - 양수 값만 허용됩니다. (`DECIMAL(10,2)`)

Response 설명

1. 성공 (`SUCCESS`)
 - `resultCode` : "SUCCESS"
 - `message` : "Module updated successfully"

- `data`: 수정된 모듈 정보 포함.

2. 실패 (`FAILURE`)

- `resultCode`: `"FAILURE"`
- `message`: `"Failed to update module"`
- `errors`: 요청 중 오류 발생 시, 필드별 오류 메시지 포함.

5. 예시

✅ 성공 요청/응답

Request

- 경로: `/admin/module/update/101`

```
{
  "moduleNfcTagId": "NFC987654",
  "moduleType": "태양광 패널",
  "moduleSize": "10x10x10",
  "moduleCost": 2500.00
}
```

Response

```
{
  "resultCode": "SUCCESS",
  "message": "Module updated successfully",
  "data": {
    "moduleId": 101,
    "moduleNfcTagId": "NFC987654",
    "moduleType": "태양광 패널",
    "moduleSize": "10x10x10",
    "moduleCost": 2500.00
  }
}
```

❌ 실패 요청/응답

Request (중복된 NFC 태그 ID 사용)

- 경로: `/admin/module/update/101`

```
{
  "moduleNfcTagId": "NFC123456",
  "moduleCost": -500
}
```

Response

```
{
  "resultCode": "FAILURE",
  "message": "Failed to update module",
  "errors": [
```

```
{
  "field": "moduleNfcTagId",
  "message": "NFC tag ID is already in use"
},
{
  "field": "moduleCost",
  "message": "Cost must be a positive number"
}
]
```

모듈 관리 정보 수정

≡ API	/admin/module/update/{moduleId}
🕒 메소드	PUT
≡ 보안	Bearer Token
☀ 상태	완료
≡ 설명	관리자가 기존에 등록된 모듈 정보를 수정합니다.

1. API 개요

- API 경로: /admin/module/update/{moduleId}
- 메서드: PUT
- 설명: 관리자가 기존에 등록된 모듈 정보를 수정합니다.
 - NFC 태그 ID, 크기, 비용 등의 정보를 업데이트할 수 있습니다.
 - 다른 모듈에서 사용 중인 NFC 태그로 변경할 수 없습니다.
- 인증 필요: ☒ Bearer Token 필요 (관리자 계정만 접근 가능)

2. Request

요청 경로

/admin/module/update/{moduleId}

- moduleId → 수정할 모듈의 ID

요청 형식

```
{
  "moduleNfcTagId": "NFC987654", // NFC 태그 ID (선택, 중복 불가)
  "moduleType": "태양광 패널", // 모듈 유형 (선택)
  "moduleSize": "10x10x10", // 모듈 크기 (선택)
  "moduleCost": 2500.00 // 모듈 비용 (선택, 양수)
}
```

요청 필드 설명

필드	타입	필수 여부	설명
moduleNfcTagId	String	선택	NFC 태그 ID (중복 불가)
moduleType	String	선택	모듈 유형 (예: "배터리 모듈", "태양광 패널")
moduleSize	String	선택	모듈 크기 (예: "10x10x10")
moduleCost	Decimal(10, 2)	선택	모듈 비용 (소수점 포함 가능, 양수)

3. Response

✅ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Module updated successfully",
}
```

```

"data": {
  "moduleId": 101,          // 수정된 모듈 ID
  "moduleNfcTagId": "NFC987654", // 수정된 NFC 태그 ID
  "moduleType": "태양광 패널", // 수정된 모듈 유형
  "moduleSize": "10x10x10", // 수정된 모듈 크기
  "moduleCost": 2500.00    // 수정된 모듈 비용
}
}

```

❌ 실패 시

```

{
  "resultCode": "FAILURE",
  "message": "Failed to update module",
  "errors": [
    {
      "field": "moduleNfcTagId",
      "message": "NFC tag ID is already in use"
    },
    {
      "field": "moduleCost",
      "message": "Cost must be a positive number"
    }
  ]
}

```

4. 설명

Request 설명

1. `moduleId` :
 - `{moduleId}` 경로에 포함된 필드로, 수정하려는 **모듈의 고유 ID**.
2. `moduleNfcTagId` :
 - NFC 태그 ID를 업데이트할 수 있습니다.
 - 다른 모듈에서 사용 중인 ID로 변경할 수 없습니다.
3. `moduleType` :
 - 모듈의 유형을 변경할 수 있습니다. (예: "배터리 모듈", "태양광 패널")
4. `moduleSize` :
 - 모듈의 크기를 변경할 수 있습니다. (예: "10x10x10")
5. `moduleCost` :
 - 모듈 비용을 업데이트할 수 있습니다.
 - 양수 값만 허용됩니다. (`DECIMAL(10,2)`)

Response 설명

1. 성공 (`SUCCESS`)
 - `resultCode` : "SUCCESS"
 - `message` : "Module updated successfully"

- `data`: 수정된 모듈 정보 포함.

2. 실패 (`FAILURE`)

- `resultCode`: `"FAILURE"`
- `message`: `"Failed to update module"`
- `errors`: 요청 중 오류 발생 시, 필드별 오류 메시지 포함.

5. 예시

✅ 성공 요청/응답

Request

- 경로: `/admin/module/update/101`

```
{
  "moduleNfcTagId": "NFC987654",
  "moduleType": "태양광 패널",
  "moduleSize": "10x10x10",
  "moduleCost": 2500.00
}
```

Response

```
{
  "resultCode": "SUCCESS",
  "message": "Module updated successfully",
  "data": {
    "moduleId": 101,
    "moduleNfcTagId": "NFC987654",
    "moduleType": "태양광 패널",
    "moduleSize": "10x10x10",
    "moduleCost": 2500.00
  }
}
```

❌ 실패 요청/응답

Request (중복된 NFC 태그 ID 사용)

- 경로: `/admin/module/update/101`

```
{
  "moduleNfcTagId": "NFC123456",
  "moduleCost": -500
}
```

Response

```
{
  "resultCode": "FAILURE",
  "message": "Failed to update module",
  "errors": [
```

```
{
  "field": "moduleNfcTagId",
  "message": "NFC tag ID is already in use"
},
{
  "field": "moduleCost",
  "message": "Cost must be a positive number"
}
]
```

모듈 관리 정보 삭제

≡ API	/admin/module/delete/{moduleId}
🕒 메소드	DELETE
≡ 보안	Bearer Token
☀ 상태	완료
≡ 설명	관리자가 기존에 등록된 모듈을 삭제할 수 있습니다.

1. API 개요

- API 경로: /admin/module/delete/{moduleId}
- 메서드: DELETE
- 설명:
 - 관리자가 기존에 등록된 모듈을 삭제할 수 있습니다.
 - 현재 대여 중인 모듈은 삭제할 수 없습니다.
- 인증 필요: ☒ Bearer Token 필요 (관리자 계정만 접근 가능)

2. Request

요청 경로

/admin/module/delete/{moduleId}

- {moduleId} → 삭제할 모듈의 ID

요청 형식

- 요청 본문 없음 (경로 파라미터에 모듈 ID 포함)

3. Response

✅ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Module deleted successfully",
  "data": {
    "moduleId": 101,           // 삭제된 모듈 ID
    "moduleNfcTagId": "NFC123456", // 삭제된 NFC 태그 ID
    "moduleType": "배터리 모듈", // 삭제된 모듈 유형
    "moduleSize": "5x5x5",      // 삭제된 모듈 크기
    "moduleCost": 1200.00      // 삭제된 모듈 비용
  }
}
```

❌ 실패 시

```
{
  "resultCode": "FAILURE",
  "message": "Failed to delete module",
}
```

```

"errors": [
  {
    "field": "moduleId",
    "message": "Module ID not found"
  },
  {
    "field": "moduleId",
    "message": "Module is currently rented and cannot be deleted"
  }
]
}

```

4. 설명

Request 설명

1. `moduleId` :
 - `{moduleId}` 경로에 포함된 필드로, 삭제하려는 **모듈의 고유 ID**.
 - 존재하지 않거나 유효하지 않은 `moduleId` 를 입력하면 삭제 실패.
2. 삭제할 수 없는 경우:
 - 현재 대여 중인 모듈은 삭제할 수 없음. (`rent_history` 테이블에서 `in-progress` 상태 확인)
 - 존재하지 않는 모듈 ID를 입력할 경우 삭제 실패.

Response 설명

1. 성공 (`SUCCESS`)
 - `resultCode` : `"SUCCESS"`
 - `message` : `"Module deleted successfully"`
 - `data` : 삭제된 모듈의 정보 포함 (`moduleId` , `moduleNfcTagId` , `moduleSize` , `moduleCost`).
2. 실패 (`FAILURE`)
 - `resultCode` : `"FAILURE"`
 - `message` : `"Failed to delete module"`
 - `errors` : 요청 중 오류 발생 시, 필드별 오류 메시지 포함.

5. 예시

✅ 성공 요청/응답

Request

- 경로: `/admin/module/delete/101`

Response

```

{
  "resultCode": "SUCCESS",
  "message": "Module deleted successfully",
  "data": {
    "moduleId": 101,
    "moduleNfcTagId": "NFC123456",
    "moduleType": "배터리 모듈",

```



```
    "moduleSize": "5x5x5",
    "moduleCost": 1200.00
  }
}
```

❌ 실패 요청/응답

Request (존재하지 않는 모듈 ID)

- 경로: `/admin/module/delete/9999`

Response

```
{
  "resultCode": "FAILURE",
  "message": "Failed to delete module",
  "errors": [
    {
      "field": "moduleId",
      "message": "Module ID not found"
    }
  ]
}
```

Request (현재 대여 중인 모듈 삭제 시도)

- 경로: `/admin/module/delete/102`

Response

```
{
  "resultCode": "FAILURE",
  "message": "Failed to delete module",
  "errors": [
    {
      "field": "moduleId",
      "message": "Module is currently rented and cannot be deleted"
    }
  ]
}
```

모듈 관리 정보 조회

≡ API	/admin/module/list
🕒 메소드	GET
≡ 보안	Bearer Token
☀ 상태	완료
≡ 설명	관리자가 시스템에 등록된 모듈 목록을 조회합니다.

1. API 개요

- API 경로: /admin/module/list
- 메서드: GET
- 설명: 관리자가 시스템에 등록된 모듈 목록을 조회합니다.
- 인증 필요: ☒ Bearer Token 필요 (관리자 계정만 접근 가능)
- 기능:
 - 모듈 이름 및 NFC 태그 검색
 - 모듈 상태별 필터링 (active, inactive, maintenance)
 - 페이지네이션 지원
 - 현재 대여 여부 및 정비 기록 포함

2. Request

요청 형식

```
{
  "search": "String",      // 모듈 이름 또는 NFC 태그 검색 키워드 (선택)
  "status": "String",      // 모듈 상태 필터 (선택, 예: "active", "inactive", "maintenanc
e")
  "page": 1,               // 페이지 번호 (선택, 기본값: 1)
  "pageSize": 10           // 페이지 크기 (선택, 기본값: 10)
}
```

요청 필드 설명

필드	타입	필수 여부	설명
search	String	✗ 선택	모듈 이름 또는 NFC 태그 검색 키워드
status	String	✗ 선택	모듈 상태 필터링 (active, inactive, maintenance)
page	Integer	✗ 선택	결과 페이지 번호 (기본값: 1)
pageSize	Integer	✗ 선택	한 페이지당 반환할 모듈 개수 (기본값: 10)

3. Response

✓ 성공 시

```
{
  "resultCode": "SUCCESS",
```

```

"message": "Module list retrieved successfully",
"data": {
  "modules": [                                // 모듈 리스트
    {
      "moduleId": 101,                        // 모듈 ID
      "moduleNfcTagId": "NFC123456", // NFC 태그 ID
      "moduleType": "배터리 모듈", // 모듈 유형
      "moduleSize": "5x5x5",           // 모듈 크기
      "moduleCost": 1500.00,           // 모듈 비용
      "status": "active",               // 모듈 상태 (active/inactive/maintenance)
      "currentRentId": 1001,           // 현재 대여 ID (없으면 null)
      "lastMaintenance": "2025-01-01", // 마지막 정비 날짜
      "lastUsed": "2025-01-10"        // 마지막 사용 날짜
    },
    {
      "moduleId": 102,
      "moduleNfcTagId": "NFC654321",
      "moduleType": "태양광 패널",
      "moduleSize": "10x10x10",
      "moduleCost": 2000.00,
      "status": "inactive",
      "currentRentId": null,
      "lastMaintenance": "2024-12-15",
      "lastUsed": null
    }
  ],
  "pagination": {                            // 페이지네이션 정보
    "currentPage": 1,
    "totalPages": 3,
    "totalItems": 25,
    "pageSize": 10
  }
}

```

❌ 실패 시

```

{
  "resultCode": "FAILURE",
  "message": "Failed to retrieve module list",
  "errors": [
    {
      "field": "status",
      "message": "Invalid status value"
    }
  ]
}

```

4. 설명

Request 설명

1. **search**:
 - 모듈 이름이나 NFC 태그를 검색 가능.

- 입력하지 않으면 모든 모듈을 반환.
2. `status` :
 - 특정 상태의 모듈만 필터링 (`active` , `inactive` , `maintenance`).
 3. `page` :
 - 결과 페이지 번호를 지정.
 4. `pageSize` :
 - 한 페이지당 반환할 모듈 개수를 설정.

Response 설명

1. 성공 (`SUCCESS`)

- `resultCode` : "SUCCESS"
- `message` : "Module list retrieved successfully"
- `data` :
 - `modules` : 조회된 모듈 목록.
 - `moduleId` : 모듈 ID.
 - `moduleNfcTagId` : NFC 태그 ID.
 - `moduleType` : 모듈 유형.
 - `moduleSize` : 모듈 크기.
 - `moduleCost` : 모듈 가격.
 - `status` : 모듈 상태 (`active` , `inactive` , `maintenance`).
 - `currentRentId` : 현재 대여 ID (`null` 일 경우 대여 중이 아님).
 - `lastMaintenance` : 마지막 정비 날짜.
 - `lastUsed` : 마지막 사용 날짜.
 - `pagination` : 페이지네이션 정보 (`currentPage` , `totalPages` , `totalItems` , `pageSize` 포함).

2. 실패 (`FAILURE`)

- `resultCode` : "FAILURE"
- `message` : "Failed to retrieve module list"
- `errors` : 요청 중 오류가 발생한 필드와 메시지 리스트.

5. 예시

✅ 성공 요청/응답

NFC 태그로 모듈 검색

```
{
  "search": "NFC123",
  "status": "active",
  "page": 1,
  "pageSize": 5
}
```

```
{
  "resultCode": "SUCCESS",
```

```

"message": "Module list retrieved successfully",
"data": {
  "modules": [
    {
      "moduleId": 101,
      "moduleNfcTagId": "NFC123456",
      "moduleType": "배터리 모듈",
      "moduleSize": "5x5x5",
      "moduleCost": 1500.00,
      "status": "active",
      "currentRentId": 1001,
      "lastMaintenance": "2025-01-01",
      "lastUsed": "2025-01-10"
    }
  ],
  "pagination": {
    "currentPage": 1,
    "totalPages": 1,
    "totalItems": 1,
    "pageSize": 5
  }
}

```

❌ 실패 요청/응답

잘못된 상태값 필터링

```

{
  "status": "invalidStatus"
}

```

```

{
  "resultCode": "FAILURE",
  "message": "Failed to retrieve module list",
  "errors": [
    {
      "field": "status",
      "message": "Invalid status value"
    }
  ]
}

```

모듈 관리 정보 등록

≡ API	/admin/module/register
🕒 메소드	POST
≡ 보안	Bearer Token
☀️ 상태	완료
≡ 설명	관리자가 새로운 모듈 정보를 시스템에 등록합니다.

1. API 개요

- API 경로: /admin/module/register
- 메서드: POST
- 설명: 관리자가 새로운 모듈 정보를 시스템에 등록합니다.
 - 각 모듈은 NFC 태그를 통해 식별됩니다.
 - 등록된 모듈은 기본적으로 inactive 상태로 설정됩니다.
- 인증 필요: ☒ Bearer Token 필요 (관리자 계정만 접근 가능)

2. Request

요청 형식

```
{
  "moduleNfcTagId": "NFC123456", // NFC 태그 ID (고유 값)
  "moduleType": "배터리 모듈", // 모듈 유형 (예: 배터리, 태양광 패널 등)
  "moduleSize": "5x5x5", // 모듈 크기
  "moduleCost": 1500.00 // 모듈 비용
}
```

요청 필드 설명

필드	타입	필수 여부	설명
moduleNfcTagId	String	☒ 필수	NFC 태그 ID (유일해야 함)
moduleType	String	☒ 필수	모듈 유형 (예: "배터리 모듈", "태양광 패널")
moduleSize	String	☒ 필수	모듈 크기 (예: "5x5x5")
moduleCost	Decimal(10,2)	☒ 필수	모듈 비용 (소수점 포함 가능)

3. Response

☒ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Module registered successfully",
  "data": {
    "moduleId": 101, // 등록된 모듈 ID
    "moduleNfcTagId": "NFC123456", // NFC 태그 ID
    "moduleType": "배터리 모듈", // 모듈 유형
    "moduleSize": "5x5x5", // 모듈 크기
  }
}
```

```

    "moduleCost": 1500.00,          // 모듈 비용
    "status": "inactive"           // 초기 모듈 상태 (기본값: inactive)
  }
}

```

❌ 실패 시

```

{
  "resultCode": "FAILURE",
  "message": "Failed to register module",
  "errors": [
    {
      "field": "moduleNfcTagId",
      "message": "NFC tag ID is already in use"
    },
    {
      "field": "moduleType",
      "message": "Invalid module type"
    }
  ]
}

```

4. 설명

Request 설명

1. `moduleNfcTagId` :
 - 모듈과 연결된 NFC 태그의 **고유 ID**.
 - 이미 등록된 `moduleNfcTagId` 는 허용되지 않음.
2. `moduleType` :
 - 모듈의 유형을 나타냄 (예: "배터리 모듈", "태양광 패널" 등).
3. `moduleSize` :
 - 모듈 크기를 나타냄.
 - 예: "5x5x5" 형식.
4. `moduleCost` :
 - 모듈의 가격을 나타냄.
 - 소수점 포함 가능 (`DECIMAL(10,2)`) 형식.

Response 설명

1. 성공 (`SUCCESS`)
 - `resultCode` : "SUCCESS"
 - `message` : "Module registered successfully"
 - `data` :
 - `moduleId` : 시스템에서 자동 생성한 모듈의 고유 ID.
 - `moduleNfcTagId` : 등록된 NFC 태그 ID.
 - `moduleType` : 등록된 모듈 유형.

- `moduleSize`: 등록된 모듈 크기.
- `moduleCost`: 등록된 모듈 비용.
- `status`: `"inactive"` (기본 상태).

2. 실패 (`FAILURE`)

- `resultCode`: `"FAILURE"`
- `message`: `"Failed to register module"`
- `errors`: 오류 발생 필드와 메시지 포함.

5. 예시

✅ 성공 요청/응답

Request

```
{
  "moduleNfcTagId": "NFC123456",
  "moduleType": "배터리 모듈",
  "moduleSize": "5x5x5",
  "moduleCost": 1500.00
}
```

Response

```
{
  "resultCode": "SUCCESS",
  "message": "Module registered successfully",
  "data": {
    "moduleId": 101,
    "moduleNfcTagId": "NFC123456",
    "moduleType": "배터리 모듈",
    "moduleSize": "5x5x5",
    "moduleCost": 1500.00,
    "status": "inactive"
  }
}
```

❌ 실패 요청/응답

Request (중복된 NFC 태그 ID)

```
{
  "moduleNfcTagId": "NFC123456",
  "moduleType": "배터리 모듈",
  "moduleSize": "5x5x5",
  "moduleCost": 1500.00
}
```

Response

```
{
  "resultCode": "FAILURE",
```



```
"message": "Failed to register module",
"errors": [
  {
    "field": "moduleNfcTagId",
    "message": "NFC tag ID is already in use"
  }
]
```

❌ 실패 요청/응답

Request (잘못된 모듈 유형)

```
{
  "moduleNfcTagId": "NFC789123",
  "moduleType": "잘못된 유형",
  "moduleSize": "5x5x5",
  "moduleCost": 1500.00
}
```

Response

```
{
  "resultCode": "FAILURE",
  "message": "Failed to register module",
  "errors": [
    {
      "field": "moduleType",
      "message": "Invalid module type"
    }
  ]
}
```

모듈 관리 정보 수정

≡ API	/admin/module/update/{moduleId}
🕒 메소드	PUT
≡ 보안	Bearer Token
☀ 상태	완료
≡ 설명	관리자가 기존에 등록된 모듈 정보를 수정합니다.

1. API 개요

- API 경로: /admin/module/update/{moduleId}
- 메서드: PUT
- 설명: 관리자가 기존에 등록된 모듈 정보를 수정합니다.
 - NFC 태그 ID, 크기, 비용 등의 정보를 업데이트할 수 있습니다.
 - 다른 모듈에서 사용 중인 NFC 태그로 변경할 수 없습니다.
- 인증 필요: ☒ Bearer Token 필요 (관리자 계정만 접근 가능)

2. Request

요청 경로

/admin/module/update/{moduleId}

- moduleId → 수정할 모듈의 ID

요청 형식

```
{
  "moduleNfcTagId": "NFC987654", // NFC 태그 ID (선택, 중복 불가)
  "moduleType": "태양광 패널", // 모듈 유형 (선택)
  "moduleSize": "10x10x10", // 모듈 크기 (선택)
  "moduleCost": 2500.00 // 모듈 비용 (선택, 양수)
}
```

요청 필드 설명

필드	타입	필수 여부	설명
moduleNfcTagId	String	선택	NFC 태그 ID (중복 불가)
moduleType	String	선택	모듈 유형 (예: "배터리 모듈", "태양광 패널")
moduleSize	String	선택	모듈 크기 (예: "10x10x10")
moduleCost	Decimal(10,2)	선택	모듈 비용 (소수점 포함 가능, 양수)

3. Response

✅ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Module updated successfully",
}
```

```

"data": {
  "moduleId": 101,          // 수정된 모듈 ID
  "moduleNfcTagId": "NFC987654", // 수정된 NFC 태그 ID
  "moduleType": "태양광 패널", // 수정된 모듈 유형
  "moduleSize": "10x10x10", // 수정된 모듈 크기
  "moduleCost": 2500.00    // 수정된 모듈 비용
}
}

```

❌ 실패 시

```

{
  "resultCode": "FAILURE",
  "message": "Failed to update module",
  "errors": [
    {
      "field": "moduleNfcTagId",
      "message": "NFC tag ID is already in use"
    },
    {
      "field": "moduleCost",
      "message": "Cost must be a positive number"
    }
  ]
}

```

4. 설명

Request 설명

1. `moduleId` :
 - `{moduleId}` 경로에 포함된 필드로, 수정하려는 **모듈의 고유 ID**.
2. `moduleNfcTagId` :
 - NFC 태그 ID를 업데이트할 수 있습니다.
 - 다른 모듈에서 사용 중인 ID로 변경할 수 없습니다.
3. `moduleType` :
 - 모듈의 유형을 변경할 수 있습니다. (예: "배터리 모듈", "태양광 패널")
4. `moduleSize` :
 - 모듈의 크기를 변경할 수 있습니다. (예: "10x10x10")
5. `moduleCost` :
 - 모듈 비용을 업데이트할 수 있습니다.
 - 양수 값만 허용됩니다. (`DECIMAL(10,2)`)

Response 설명

1. 성공 (`SUCCESS`)
 - `resultCode` : `"SUCCESS"`
 - `message` : `"Module updated successfully"`

- `data`: 수정된 모듈 정보 포함.

2. 실패 (`FAILURE`)

- `resultCode`: `"FAILURE"`
- `message`: `"Failed to update module"`
- `errors`: 요청 중 오류 발생 시, 필드별 오류 메시지 포함.

5. 예시

✅ 성공 요청/응답

Request

- 경로: `/admin/module/update/101`

```
{
  "moduleNfcTagId": "NFC987654",
  "moduleType": "태양광 패널",
  "moduleSize": "10x10x10",
  "moduleCost": 2500.00
}
```

Response

```
{
  "resultCode": "SUCCESS",
  "message": "Module updated successfully",
  "data": {
    "moduleId": 101,
    "moduleNfcTagId": "NFC987654",
    "moduleType": "태양광 패널",
    "moduleSize": "10x10x10",
    "moduleCost": 2500.00
  }
}
```

❌ 실패 요청/응답

Request (중복된 NFC 태그 ID 사용)

- 경로: `/admin/module/update/101`

```
{
  "moduleNfcTagId": "NFC123456",
  "moduleCost": -500
}
```

Response

```
{
  "resultCode": "FAILURE",
  "message": "Failed to update module",
  "errors": [
```

```
{
  "field": "moduleNfcTagId",
  "message": "NFC tag ID is already in use"
},
{
  "field": "moduleCost",
  "message": "Cost must be a positive number"
}
]
```

모듈 관리 정보 삭제

≡ API	/admin/module/delete/{moduleId}
🕒 메소드	DELETE
≡ 보안	Bearer Token
☀ 상태	완료
≡ 설명	관리자가 기존에 등록된 모듈을 삭제할 수 있습니다.

1. API 개요

- API 경로: /admin/module/delete/{moduleId}
- 메서드: DELETE
- 설명:
 - 관리자가 기존에 등록된 모듈을 삭제할 수 있습니다.
 - 현재 대여 중인 모듈은 삭제할 수 없습니다.
- 인증 필요: ☒ Bearer Token 필요 (관리자 계정만 접근 가능)

2. Request

요청 경로

/admin/module/delete/{moduleId}

- {moduleId} → 삭제할 모듈의 ID

요청 형식

- 요청 본문 없음 (경로 파라미터에 모듈 ID 포함)

3. Response

✅ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Module deleted successfully",
  "data": {
    "moduleId": 101,           // 삭제된 모듈 ID
    "moduleNfcTagId": "NFC123456", // 삭제된 NFC 태그 ID
    "moduleType": "배터리 모듈", // 삭제된 모듈 유형
    "moduleSize": "5x5x5",      // 삭제된 모듈 크기
    "moduleCost": 1200.00      // 삭제된 모듈 비용
  }
}
```

❌ 실패 시

```
{
  "resultCode": "FAILURE",
  "message": "Failed to delete module",
}
```

```

"errors": [
  {
    "field": "moduleId",
    "message": "Module ID not found"
  },
  {
    "field": "moduleId",
    "message": "Module is currently rented and cannot be deleted"
  }
]
}

```

4. 설명

Request 설명

1. `moduleId` :
 - `{moduleId}` 경로에 포함된 필드로, 삭제하려는 **모듈의 고유 ID**.
 - 존재하지 않거나 유효하지 않은 `moduleId` 를 입력하면 삭제 실패.
2. 삭제할 수 없는 경우:
 - 현재 대여 중인 모듈은 삭제할 수 없음. (`rent_history` 테이블에서 `in-progress` 상태 확인)
 - 존재하지 않는 모듈 ID를 입력할 경우 삭제 실패.

Response 설명

1. 성공 (`SUCCESS`)
 - `resultCode` : `"SUCCESS"`
 - `message` : `"Module deleted successfully"`
 - `data` : 삭제된 모듈의 정보 포함 (`moduleId` , `moduleNfcTagId` , `moduleSize` , `moduleCost`).
2. 실패 (`FAILURE`)
 - `resultCode` : `"FAILURE"`
 - `message` : `"Failed to delete module"`
 - `errors` : 요청 중 오류 발생 시, 필드별 오류 메시지 포함.

5. 예시

✅ 성공 요청/응답

Request

- 경로: `/admin/module/delete/101`

Response

```

{
  "resultCode": "SUCCESS",
  "message": "Module deleted successfully",
  "data": {
    "moduleId": 101,
    "moduleNfcTagId": "NFC123456",
    "moduleType": "배터리 모듈",

```

```
    "moduleSize": "5x5x5",  
    "moduleCost": 1200.00  
  }  
}
```

❌ 실패 요청/응답

Request (존재하지 않는 모듈 ID)

- 경로: `/admin/module/delete/9999`

Response

```
{  
  "resultCode": "FAILURE",  
  "message": "Failed to delete module",  
  "errors": [  
    {  
      "field": "moduleId",  
      "message": "Module ID not found"  
    }  
  ]  
}
```

Request (현재 대여 중인 모듈 삭제 시도)

- 경로: `/admin/module/delete/102`

Response

```
{  
  "resultCode": "FAILURE",  
  "message": "Failed to delete module",  
  "errors": [  
    {  
      "field": "moduleId",  
      "message": "Module is currently rented and cannot be deleted"  
    }  
  ]  
}
```


옵션 관리 정보 조회

≡ API	/admin/option/list
🕒 메소드	GET
≡ 보안	Bearer Token
☀ 상태	완료
≡ 설명	관리자가 등록된 옵션 목록을 조회합니다.

. API 개요

- API 경로: /admin/option/list
- 메서드: GET
- 설명:
 - 관리자가 등록된 옵션 목록을 조회합니다.
 - 이름 검색, 상태 필터링, 페이지네이션 및 옵션 이미지 제공.
- 인증 필요: ☒ Bearer Token 필요 (관리자 계정만 접근 가능)

2. Request

요청 경로

/admin/option/list

요청 형식

```
{
  "search": "String",      // 옵션 이름 검색 키워드 (선택)
  "status": "String",      // 옵션 상태 필터 (선택, 예: "active", "inactive")
  "page": 1,               // 페이지 번호 (선택, 기본값: 1)
  "pageSize": 10           // 페이지 크기 (선택, 기본값: 10)
}
```

요청 필드 설명

필드	타입	필수 여부	설명
search	String	❌ 선택	옵션 이름 검색 키워드
status	String	❌ 선택	active 또는 inactive 상태 필터링
page	Integer	❌ 선택	결과 페이지 번호 (기본값: 1)
pageSize	Integer	❌ 선택	한 페이지당 반환할 옵션 개수 (기본값: 10)

3. Response

✅ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Options retrieved successfully",
  "data": {
    "options": [
```

```

{
  "optionId": 1,
  "optionName": "침대",
  "optionType": "기본",
  "optionSize": "2x3",
  "optionCost": 500.00,
  "stockQuantity": 10,
  "status": "active",
  "description": "차량 내에서 사용할 수 있는 침대 옵션.",
  "imgUrls": [
    "https://example.com/images/bed1.jpg",
    "https://example.com/images/bed2.jpg"
  ]
},
{
  "optionId": 2,
  "optionName": "냉장고",
  "optionType": "가전",
  "optionSize": "1x1",
  "optionCost": 800.00,
  "stockQuantity": 5,
  "status": "inactive",
  "description": "신선 식품을 저장하기 위한 냉장고.",
  "imgUrls": [
    "https://example.com/images/fridge1.jpg"
  ]
}
],
"pagination": {
  "currentPage": 1,
  "totalPages": 3,
  "totalItems": 25,
  "pageSize": 10
}
}
}

```

❌ 실패 시

```

{
  "resultCode": "FAILURE",
  "message": "Failed to retrieve options",
  "errors": [
    {
      "field": "search",
      "message": "Invalid search value"
    }
  ]
}

```

4. 설명

Request 설명

1. `search` (옵션)
 - 특정 옵션을 검색할 때 사용.
 - 예: "침대", "냉장고", "배터리"
 - 입력하지 않으면 모든 옵션을 반환.
2. `status` (옵션)
 - 특정 상태의 옵션만 조회.
 - 예: "active" 또는 "inactive"
3. `page` (옵션)
 - 조회할 페이지 번호 (기본값: 1).
4. `pageSize` (옵션)
 - 한 페이지당 옵션 개수 (기본값: 10).

Response 설명

1. `options` :
 - 조회된 옵션들의 리스트.
 - 각 옵션은 `optionId`, `optionName`, `optionType`, `optionSize`, `optionCost`, `stockQuantity`, `status`, `description`, `imgUrls` 정보를 포함.
2. `pagination` :
 - 페이지네이션 관련 정보.
 - 현재 페이지, 총 페이지 수, 총 항목 수, 페이지 크기를 포함.

5. 예시

✅ 성공 요청/응답

Request

```
{
  "search": "배터리",
  "status": "active",
  "page": 1,
  "pageSize": 5
}
```

Response

```
{
  "resultCode": "SUCCESS",
  "message": "Options retrieved successfully",
  "data": {
    "options": [
      {
        "optionId": 3,
        "optionName": "배터리 팩",
        "optionType": "전력",
        "optionSize": "1x1",
        "optionCost": 300.00,
        "stockQuantity": 20,
```

```
    "status": "active",
    "description": "전력 공급용 배터리.",
    "imgUrls": [
      "https://example.com/images/battery1.jpg"
    ]
  },
  "pagination": {
    "currentPage": 1,
    "totalPages": 1,
    "totalItems": 1,
    "pageSize": 5
  }
}
```

❌ 실패 요청/응답

Request

```
{
  "search": "잘못된 검색어"
}
```

Response

```
{
  "resultCode": "FAILURE",
  "message": "Failed to retrieve options",
  "errors": [
    {
      "field": "search",
      "message": "Invalid search value"
    }
  ]
}
```

옵션 관리 정보 등록

≡ API	/admin/option/register
🕒 메소드	POST
≡ 보안	Bearer Token
☀️ 상태	완료
≡ 설명	관리자가 새로운 옵션을 등록합니다.

1. API 개요

- API 경로: /admin/option/register
- 메서드: POST
- 설명:
 - 관리자가 새로운 옵션을 등록합니다.
 - 옵션의 크기, 비용, 재고, 이미지 및 설명 포함.
- 인증 필요: ☒ Bearer Token 필요 (관리자 계정만 접근 가능)

2. Request

요청 경로

/admin/option/create

요청 형식

```
{
  "optionName": "String",           // 옵션 이름 (필수, 중복 불가)
  "optionType": "String",          // 옵션 유형 (필수, 예: "전력", "가전")
  "optionSize": "String",          // 옵션 크기 (필수, 예: "2x3")
  "optionCost": "Decimal",         // 옵션 비용 (필수, 0 이상)
  "stockQuantity": "Integer",      // 옵션 재고 수량 (필수, 0 이상)
  "status": "String",              // 옵션 상태 (필수, 예: "active", "inactive")
  "description": "String",         // 옵션 설명 (선택)
  "imgUrls": ["String"]           // 옵션 이미지 URL 리스트 (선택)
}
```

요청 필드 설명

필드	타입	필수 여부	설명
optionName	String	☒ 필수	옵션 이름 (중복 불가)
optionType	String	☒ 필수	옵션 유형 ("전력", "가전", "기분", "기타")
optionSize	String	☒ 필수	옵션 크기 (예: "2x3")
optionCost	Decimal	☒ 필수	옵션 비용 (0 이상)
stockQuantity	Integer	☒ 필수	옵션 재고 수량 (0 이상)
status	String	☒ 필수	"active" 또는 "inactive" 상태 설정
description	String	☒ 선택	옵션 설명
imgUrls	Array[String]	☒ 선택	옵션 이미지 URL 리스트

3. Response

✅ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Option registered successfully",
  "data": {
    "optionId": 101,
    "optionName": "배터리 팩",
    "optionType": "전력",
    "optionSize": "1x1",
    "optionCost": 15000.00,
    "stockQuantity": 30,
    "status": "active",
    "description": "전력 공급을 위한 배터리 팩",
    "imgUrls": [
      "https://example.com/images/battery1.jpg",
      "https://example.com/images/battery2.jpg"
    ]
  }
}
```

❌ 실패 시

```
{
  "resultCode": "FAILURE",
  "message": "Failed to register option",
  "errors": [
    {
      "field": "optionName",
      "message": "Option name already exists"
    },
    {
      "field": "optionCost",
      "message": "Cost must be a positive number"
    },
    {
      "field": "stockQuantity",
      "message": "Stock must be a positive integer"
    }
  ]
}
```

4. 설명

Request 설명

1. `optionName` (필수)
 - 옵션의 이름 (고유해야 함).
 - 중복된 이름은 허용되지 않음.
2. `optionType` (필수)

- 옵션 유형을 지정 ("전력", "가전", "기본", "기타" 등).
3. `optionSize` (필수)
 - 옵션이 차지하는 공간 크기를 입력 (예: "2x3").
 4. `optionCost` (필수)
 - 옵션의 가격을 설정 (0 이상).
 5. `stockQuantity` (필수)
 - 옵션의 재고 수량을 설정 (0 이상).
 6. `status` (필수)
 - "active" 또는 "inactive" 중 하나를 설정.
 7. `description` (선택)
 - 옵션에 대한 추가 설명.
 8. `imgUrls` (선택)
 - 옵션 관련 이미지 URL 리스트.

Response 설명

1. 성공 (`SUCCESS`)

- `resultCode`: "SUCCESS"
- `message`: "Option registered successfully"
- `data`: 등록된 옵션 정보 (`optionId`, `optionName`, `optionType`, `optionSize`, `optionCost`, `stockQuantity`, `status`, `description`, `imgUrls` 포함).

2. 실패 (`FAILURE`)

- `resultCode`: "FAILURE"
- `message`: "Failed to register option"
- `errors`: 오류가 발생한 필드와 메시지 리스트.

5. 예시

✅ 성공 요청/응답

Request

```
{
  "optionName": "태양광 패널",
  "optionType": "전력",
  "optionSize": "2x1",
  "optionCost": 25000.00,
  "stockQuantity": 15,
  "status": "active",
  "description": "친환경 에너지를 제공하는 태양광 패널",
  "imgUrls": [
    "https://example.com/images/solar1.jpg",
    "https://example.com/images/solar2.jpg"
  ]
}
```

Response

```
{
  "resultCode": "SUCCESS",
  "message": "Option registered successfully",
  "data": {
    "optionId": 102,
    "optionName": "태양광 패널",
    "optionType": "전력",
    "optionSize": "2x1",
    "optionCost": 25000.00,
    "stockQuantity": 15,
    "status": "active",
    "description": "친환경 에너지를 제공하는 태양광 패널",
    "imgUrls": [
      "https://example.com/images/solar1.jpg",
      "https://example.com/images/solar2.jpg"
    ]
  }
}
```

❌ 실패 요청/응답

Request

```
{
  "optionName": "태양광 패널",
  "optionType": "전력",
  "optionSize": "2x1",
  "optionCost": -5000,
  "stockQuantity": -2,
  "status": "active"
}
```

Response

```
{
  "resultCode": "FAILURE",
  "message": "Failed to register option",
  "errors": [
    {
      "field": "optionName",
      "message": "Option name already exists"
    },
    {
      "field": "optionCost",
      "message": "Cost must be a positive number"
    },
    {
      "field": "stockQuantity",
      "message": "Stock must be a positive integer"
    }
  ]
}
```


옵션 관리 정보 수정

≡ API	/admin/option/update/{optionId}
🕒 메소드	PUT
≡ 보안	Bearer Token
☀ 상태	완료
≡ 설명	관리자가 등록된 옵션 정보를 수정합니다.

1. API 개요

- API 경로: /admin/option/update/{optionId}
- 메서드: PUT
- 설명:
 - 관리자가 등록된 옵션 정보를 수정합니다.
 - 옵션의 크기, 비용, 재고, 이미지 및 설명을 포함.
- 인증 필요: ☒ Bearer Token 필요 (관리자 계정만 접근 가능)

2. Request

요청 경로

/admin/option/update/{optionId}

요청 형식

```
{
  "optionName": "String",           // 옵션 이름 (선택, 중복 불가)
  "optionType": "String",          // 옵션 유형 (선택, 예: "전력", "가전")
  "optionSize": "String",          // 옵션 크기 (선택, 예: "2x3")
  "optionCost": "Decimal",         // 옵션 비용 (선택, 0 이상)
  "stockQuantity": "Integer",      // 옵션 재고 수량 (선택, 0 이상)
  "status": "String",              // 옵션 상태 (선택, 예: "active", "inactive")
  "description": "String",         // 옵션 설명 (선택)
  "imgUrls": ["String"]           // 옵션 이미지 URL 리스트 (선택)
}
```

요청 필드 설명

필드	타입	필수 여부	설명
optionName	String	❌ 선택	옵션 이름 (중복 불가)
optionType	String	❌ 선택	옵션 유형 ("전력" , "가전" , "기분" , "기타")
optionSize	String	❌ 선택	옵션 크기 (예: "2x3")
optionCost	Decimal	❌ 선택	옵션 비용 (0 이상)
stockQuantity	Integer	❌ 선택	옵션 재고 수량 (0 이상)
status	String	❌ 선택	"active" 또는 "inactive" 상태 설정
description	String	❌ 선택	옵션에 대한 설명
imgUrls	Array[String]	❌ 선택	옵션 이미지 URL 리스트

3. Response

✅ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Option updated successfully",
  "data": {
    "optionId": 101,
    "optionName": "배터리 팩",
    "optionType": "전력",
    "optionSize": "1x1",
    "optionCost": 15000.00,
    "stockQuantity": 30,
    "status": "active",
    "description": "전력 공급을 위한 배터리 팩",
    "imgUrls": [
      "https://example.com/images/battery1.jpg",
      "https://example.com/images/battery2.jpg"
    ]
  }
}
```

❌ 실패 시

```
{
  "resultCode": "FAILURE",
  "message": "Failed to update option",
  "errors": [
    {
      "field": "optionName",
      "message": "Option name already exists"
    },
    {
      "field": "optionCost",
      "message": "Cost must be a positive number"
    },
    {
      "field": "stockQuantity",
      "message": "Stock must be a positive integer"
    }
  ]
}
```

4. 설명

Request 설명

1. `optionName` (선택)
 - 옵션의 이름 (고유해야 함).
 - 중복된 이름은 허용되지 않음.
2. `optionType` (선택)

- 옵션 유형을 지정 ("전력", "가전", "기본", "기타" 등).
3. `optionSize` (선택)
 - 옵션이 차지하는 공간 크기를 입력 (예: "2x3").
 4. `optionCost` (선택)
 - 옵션의 가격을 설정 (0 이상).
 5. `stockQuantity` (선택)
 - 옵션의 재고 수량을 설정 (0 이상).
 6. `status` (선택)
 - "active" 또는 "inactive" 중 하나를 설정.
 7. `description` (선택)
 - 옵션에 대한 추가 설명.
 8. `imgUrls` (선택)
 - 옵션 관련 이미지 URL 리스트.

Response 설명

1. 성공 (`SUCCESS`)

- `resultCode`: "SUCCESS"
- `message`: "Option updated successfully"
- `data`: 수정된 옵션 정보 (`optionId`, `optionName`, `optionType`, `optionSize`, `optionCost`, `stockQuantity`, `status`, `description`, `imgUrls` 포함).

2. 실패 (`FAILURE`)

- `resultCode`: "FAILURE"
- `message`: "Failed to update option"
- `errors`: 오류가 발생한 필드와 메시지 리스트.

5. 예시

✅ 성공 요청/응답

Request

```
{
  "optionName": "태양광 패널",
  "optionType": "전력",
  "optionSize": "2x1",
  "optionCost": 25000.00,
  "stockQuantity": 15,
  "status": "active",
  "description": "친환경 에너지를 제공하는 태양광 패널",
  "imgUrls": [
    "https://example.com/images/solar1.jpg",
    "https://example.com/images/solar2.jpg"
  ]
}
```

Response

```
{
  "resultCode": "SUCCESS",
  "message": "Option updated successfully",
  "data": {
    "optionId": 102,
    "optionName": "태양광 패널",
    "optionType": "전력",
    "optionSize": "2x1",
    "optionCost": 25000.00,
    "stockQuantity": 15,
    "status": "active",
    "description": "친환경 에너지를 제공하는 태양광 패널",
    "imgUrls": [
      "https://example.com/images/solar1.jpg",
      "https://example.com/images/solar2.jpg"
    ]
  }
}
```

❌ 실패 요청/응답

Request

```
{
  "optionName": "태양광 패널",
  "optionType": "전력",
  "optionSize": "2x1",
  "optionCost": -5000,
  "stockQuantity": -2,
  "status": "active"
}
```

Response

```
{
  "resultCode": "FAILURE",
  "message": "Failed to update option",
  "errors": [
    {
      "field": "optionName",
      "message": "Option name already exists"
    },
    {
      "field": "optionCost",
      "message": "Cost must be a positive number"
    },
    {
      "field": "stockQuantity",
      "message": "Stock must be a positive integer"
    }
  ]
}
```

옵션 관리 정보 삭제

≡ API	/admin/option/delete/{optionId}
🕒 메소드	DELETE
≡ 보안	Bearer Token
☀ 상태	완료
≡ 설명	옵션의 정보 삭제

1. API 개요

- API 경로: /admin/option/delete/{optionId}
- 메서드: DELETE
- 설명:
 - 관리자가 등록된 옵션을 삭제합니다.
 - 다른 모듈, 대여 기록, 또는 모듈 세트에서 사용 중이라면 삭제할 수 없음.
- 인증 필요:  Bearer Token 필요 (관리자 계정만 접근 가능)

2. Request

요청 경로

/admin/option/delete/{optionId}

요청 형식

- 요청 본문 없음 (옵션 ID는 URL 경로로 전달)

3. Response

성공 시

```
json
복사편집
{
  "resultCode": "SUCCESS",
  "message": "Option deleted successfully",
  "data": {
    "optionId": 101,
    "optionName": "배터리 팩",
    "optionType": "전력",
    "optionSize": "1x1",
    "optionCost": 15000.00,
    "stockQuantity": 30,
    "status": "inactive",
    "description": "전력 공급을 위한 배터리 팩",
    "imgUrls": [
      "https://example.com/images/battery1.jpg",
      "https://example.com/images/battery2.jpg"
    ]
  }
}
```

```
}
```

❌ 실패 시

```
json
복사편집
{
  "resultCode": "FAILURE",
  "message": "Failed to delete option",
  "errors": [
    {
      "field": "optionId",
      "message": "Option not found"
    },
    {
      "field": "optionId",
      "message": "Option is currently in use and cannot be deleted"
    }
  ]
}
```

4. 설명

Request 설명

1. `optionId` (필수)
 - 삭제할 옵션의 고유 ID를 URL 경로로 전달.

Response 설명

1. 성공 (`SUCCESS`)
 - `resultCode` : `"SUCCESS"`
 - `message` : `"Option deleted successfully"`
 - `data` : 삭제된 옵션 정보 (`optionId` , `optionName` , `optionType` , `optionSize` , `optionCost` , `stockQuantity` , `status` , `description` , `imgUrls` 포함).
2. 실패 (`FAILURE`)
 - `resultCode` : `"FAILURE"`
 - `message` : `"Failed to delete option"`
 - `errors` : 오류가 발생한 필드와 메시지 리스트.
 - `Option not found` : 존재하지 않는 옵션 ID로 요청한 경우.
 - `Option is currently in use` : 해당 옵션이 다른 모듈, 대여 기록, 또는 모듈 세트에서 사용 중이라 삭제할 수 없는 경우.

5. 예시

✅ 성공 요청/응답

Request

- URL: `/admin/option/delete/101`

Response

```
json
복사편집
{
  "resultCode": "SUCCESS",
  "message": "Option deleted successfully",
  "data": {
    "optionId": 101,
    "optionName": "배터리 팩",
    "optionType": "전력",
    "optionSize": "1x1",
    "optionCost": 15000.00,
    "stockQuantity": 30,
    "status": "inactive",
    "description": "전력 공급을 위한 배터리 팩",
    "imgUrls": [
      "https://example.com/images/battery1.jpg",
      "https://example.com/images/battery2.jpg"
    ]
  }
}
```

❌ 실패 요청/응답

Request

- URL: `/admin/option/delete/99999` (존재하지 않는 옵션 ID)

Response (옵션 ID가 없는 경우)

```
json
복사편집
{
  "resultCode": "FAILURE",
  "message": "Failed to delete option",
  "errors": [
    {
      "field": "optionId",
      "message": "Option not found"
    }
  ]
}
```

Response (옵션이 사용 중인 경우)

```
json
복사편집
{
  "resultCode": "FAILURE",
```

```
"message": "Failed to delete option",
"errors": [
  {
    "field": "optionId",
    "message": "Option is currently in use and cannot be deleted"
  }
]
```


대여 로그 조회

≡ API	/admin/rent/list
🕒 메소드	GET
≡ 보안	Bearer Token
☀ 상태	완료
≡ 설명	관리자가 시스템 내 모든 대여 기록을 조회합니다.

1. API 개요

- API 경로: /admin/rent/list
- 메서드: GET
- 설명:
 - 관리자가 시스템 내 모든 대여 기록을 조회합니다.
 - 특정 사용자(`userId`), 특정 차량(`carId`), 특정 기간(`startDate` , `endDate`)을 기준으로 필터링할 수 있습니다.
- 인증 필요: ☒ Bearer Token 필요 (관리자 계정만 접근 가능)

2. Request

요청 형식

```
{
  "userId": "user123",          // 특정 사용자의 대여 기록 조회 (선택)
  "carId": "CAR001",           // 특정 차량의 대여 기록 조회 (선택)
  "startDate": "2025-01-01",   // 조회 시작 날짜 (선택)
  "endDate": "2025-01-31",     // 조회 종료 날짜 (선택)
  "page": 1,                   // 페이지 번호 (기본값: 1)
  "pageSize": 10               // 페이지 크기 (기본값: 10)
}
```

3. Response

☒ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Rent logs retrieved successfully",
  "data": {
    "rentLogs": [
      {
        "rentId": 101,
        "user": {
          "userId": "user123",
          "userName": "김철수",
          "userEmail": "user@example.com",
          "userPhone": "010-1234-5678"
        },
        "vehicle": {
```

```

        "vehicleId": 50,
        "vin": "VIN123456789",
        "vehicleNumber": "PBV-1234"
    },
    "modules": [
        {
            "moduleId": 301,
            "moduleName": "배터리 팩",
            "quantity": 1
        },
        {
            "moduleId": 302,
            "moduleName": "냉장고",
            "quantity": 2
        }
    ],
    "rentalPeriod": {
        "startDate": "2025-01-01",
        "endDate": "2025-01-05"
    },
    "rentCost": 500,
    "distanceTravelled": 120,
    "status": "completed",
    "createdAt": "2025-01-01T08:00:00Z",
    "updatedAt": "2025-01-05T18:00:00Z"
    }
],
"pagination": {
    "currentPage": 1,
    "totalPages": 5,
    "totalItems": 50,
    "pageSize": 10
}
}
}

```

❌ 실패 시

```

{
    "resultCode": "FAILURE",
    "message": "Failed to retrieve rent logs",
    "errors": [
        {
            "field": "userId",
            "message": "User ID not found"
        }
    ]
}

```

4. 설명

Request 설명

1. `userId` (선택)

- 특정 사용자의 대여 기록을 조회할 때 사용.
2. `carId` (선택)
 - 특정 차량의 대여 기록을 조회할 때 사용.
 3. `startDate`, `endDate` (선택)
 - 특정 기간 내의 대여 기록을 조회할 때 사용.
 4. `page` (선택)
 - 페이지 번호 (기본값: `1`).
 5. `pageSize` (선택)
 - 한 페이지당 반환할 대여 기록 수 (기본값: `10`).

Response 설명

1. 성공 (`resultCode = "SUCCESS"`)
 - `rentLogs`: 대여 기록 리스트 (각 대여 기록은 아래 정보를 포함)
 - `rentId`: 대여 ID
 - `user`: 대여한 사용자 정보 (`userId`, `userName`, `userEmail`, `userPhone`)
 - `vehicle`: 대여된 차량 정보 (`vehicleId`, `vin`, `vehicleNumber`)
 - `modules`: 대여된 모듈 리스트 (`moduleId`, `moduleName`, `quantity`)
 - `rentalPeriod`: 대여 시작일과 종료일
 - `rentCost`: 총 대여 요금
 - `distanceTravelled`: 주행 거리 (km)
 - `status`: 대여 상태 (`reserved`, `in-progress`, `completed`, `canceled`)
 - `createdAt`, `updatedAt`: 대여 기록 생성 및 수정 시간
 - `pagination`: 페이지네이션 정보 (`currentPage`, `totalPages`, `totalItems`, `pageSize`)
2. 실패 (`resultCode = "FAILURE"`)
 - 잘못된 `userId`, `carId` 등으로 조회할 수 없을 경우 `"User ID not found"`

5. 예시

✓ 성공 요청/응답

Request

```
Authorization: Bearer <token>
```

Response

```
{
  "resultCode": "SUCCESS",
  "message": "Rent logs retrieved successfully",
  "data": {
    "rentLogs": [
      {
        "rentId": 101,
        "user": {
          "userId": "user123",
```

```

        "userName": "김철수",
        "userEmail": "user@example.com",
        "userPhone": "010-1234-5678"
    },
    "vehicle": {
        "vehicleId": 50,
        "vin": "VIN123456789",
        "vehicleNumber": "PBV-1234"
    },
    "modules": [
        {
            "moduleId": 301,
            "moduleName": "배터리 팩",
            "quantity": 1
        },
        {
            "moduleId": 302,
            "moduleName": "냉장고",
            "quantity": 2
        }
    ],
    "rentalPeriod": {
        "startDate": "2025-01-01",
        "endDate": "2025-01-05"
    },
    "rentCost": 500,
    "distanceTravelled": 120,
    "status": "completed",
    "createdAt": "2025-01-01T08:00:00Z",
    "updatedAt": "2025-01-05T18:00:00Z"
    }
},
"pagination": {
    "currentPage": 1,
    "totalPages": 1,
    "totalItems": 1,
    "pageSize": 5
}
}
}

```

❌ 실패 요청/응답

Request

```
Authorization: Bearer <token>
```

Response (사용자가 존재하지 않을 경우)

```

{
    "resultCode": "FAILURE",
    "message": "Failed to retrieve rent logs",
    "errors": [
        {

```

```
    "field": "userId",  
    "message": "User ID not found"  
  }  
]  
}
```

출발지 이동 차량 로그

≡ API	/admin/rent/autonomousDriving/{rentId}
🕒 메소드	GET
≡ 보안	Bearer Token
☀ 상태	완료
≡ 설명	특정 대여 ID에 대한 자율주행 로그의 상세 데이터를 조회합니다.

1. API 개요

- API 경로: /admin/rent/autonomousDriving/{rentId}
- 메서드: GET
- 설명:
 - 특정 대여 ID에 대한 자율주행 로그의 상세 데이터를 조회합니다.
 - 자율주행 영상, 에러 로그, 차량 이동 정보 포함
- 인증 필요: ☒ Bearer Token 필요 (관리자 계정만 접근 가능)

2. Request

요청 경로

- URL: /admin/autonomousDriving/logs/{rentId}
- 경로 변수: {rentId} → 조회할 대여 ID

3. Response

✅ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Autonomous driving log retrieved successfully",
  "data": {
    "rentId": 101,
    "user": {
      "userId": "user123",
      "userName": "김철수",
      "userEmail": "user@example.com",
      "userPhone": "010-1234-5678"
    },
    "vehicle": {
      "vehicleId": 50,
      "vin": "VIN123456789",
      "vehicleNumber": "PBV-1234"
    },
    "rentStartDate": "2025-01-10",
    "rentEndDate": "2025-01-15",
    "autonomousVideos": [
      {
        "videoId": 301,
```

```

        "videoUrl": "https://example.com/video1.mp4",
        "recordedAt": "2025-01-11T14:30:00Z"
    },
    {
        "videoId": 302,
        "videoUrl": "https://example.com/video2.mp4",
        "recordedAt": "2025-01-12T10:00:00Z"
    }
],
"errorLogs": [
    {
        "errorId": 201,
        "timestamp": "2025-01-12T14:30:00Z",
        "errorCode": "SENSOR_FAILURE",
        "message": "Sensor malfunction detected"
    },
    {
        "errorId": 202,
        "timestamp": "2025-01-13T09:15:00Z",
        "errorCode": "GPS_LOST",
        "message": "GPS signal lost"
    }
],
"destination": {
    "latitude": 37.7749,
    "longitude": -122.4194
},
"status": "completed",
"totalDistance": 120,
"totalCost": 50000
}
}

```

❌ 실패 시

```

{
    "resultCode": "FAILURE",
    "message": "Failed to retrieve autonomous driving log",
    "errors": [
        {
            "field": "rentId",
            "message": "Rental ID not found"
        }
    ]
}

```

4. 설명

Request 설명

1. `rentId` (필수)

- 특정 대여 ID에 대한 자율주행 로그를 조회.
- URL 경로 변수 `{rentId}` 로 전달.

Response 설명

1. 성공 (`resultCode = "SUCCESS"`)

- `rentId` : 조회된 대여 ID
- `user` : 해당 대여를 한 사용자 정보 (`userId` , `userName` , `userEmail` , `userPhone`)
- `vehicle` : 해당 대여에 사용된 차량 정보 (`vehicleId` , `vin` , `vehicleNumber`)
- `rentStartDate` , `rentEndDate` : 대여 기간
- `autonomousVideos` : 자율주행 영상 리스트 (각 영상의 `videoId` , `videoUrl` , `recordedAt` 포함)
- `errorLogs` : 자율주행 중 발생한 오류 리스트 (각 오류의 `errorId` , `timestamp` , `errorCode` , `message` 포함)
- `destination` : 목적지 GPS 좌표 (`latitude` , `longitude`)
- `status` : 자율주행 상태 (`in-progress` 또는 `completed`)
- `totalDistance` : 총 주행 거리 (km)
- `totalCost` : 총 대여 비용 (원)

2. 실패 (`resultCode = "FAILURE"`)

- `rentId` 가 존재하지 않을 경우 → `"Rental ID not found"`

5. 예시

✅ 성공 요청/응답

Request

- URL: `/admin/autonomousDriving/logs/101`

Response

```
{
  "resultCode": "SUCCESS",
  "message": "Autonomous driving log retrieved successfully",
  "data": {
    "rentId": 101,
    "user": {
      "userId": "user123",
      "userName": "김철수",
      "userEmail": "user@example.com",
      "userPhone": "010-1234-5678"
    },
    "vehicle": {
      "vehicleId": 50,
      "vin": "VIN123456789",
      "vehicleNumber": "PBV-1234"
    },
    "rentStartDate": "2025-01-10",
    "rentEndDate": "2025-01-15",
    "autonomousVideos": [
      {
        "videoId": 301,
        "videoUrl": "https://example.com/video1.mp4",
        "recordedAt": "2025-01-11T14:30:00Z"
      }
    ]
  }
}
```



```

    {
      "videoId": 302,
      "videoUrl": "https://example.com/video2.mp4",
      "recordedAt": "2025-01-12T10:00:00Z"
    }
  ],
  "errorLogs": [
    {
      "errorId": 201,
      "timestamp": "2025-01-12T14:30:00Z",
      "errorCode": "SENSOR_FAILURE",
      "message": "Sensor malfunction detected"
    },
    {
      "errorId": 202,
      "timestamp": "2025-01-13T09:15:00Z",
      "errorCode": "GPS_LOST",
      "message": "GPS signal lost"
    }
  ],
  "destination": {
    "latitude": 37.7749,
    "longitude": -122.4194
  },
  "status": "completed",
  "totalDistance": 120,
  "totalCost": 50000
}

```

❌ 실패 요청/응답

Request

- URL: `/admin/autonomousDriving/logs/9999`

Response (대여 ID가 없는 경우)

```

{
  "resultCode": "FAILURE",
  "message": "Failed to retrieve autonomous driving log",
  "errors": [
    {
      "field": "rentId",
      "message": "Rental ID not found"
    }
  ]
}

```

모듈 장착 영상 확인

≡ API	/admin/rent/moduleVideo/{rentId}
🕒 메소드	GET
≡ 보안	Bearer Token
☀ 상태	완료
≡ 설명	대여 로그에서 해당 렌트 기록의 모듈 장착 영상을 확인(모달/팝업)

1. API 개요

- **API 경로:** /admin/rent/moduleVideo/{rentId}
- **메서드:** GET
- **설명:** 특정 대여 ID와 연결된 **모듈 장착 영상**을 확인합니다.
- **인증 필요:** Bearer Token 을 사용하여 인증.
- **기능:**
 - 대여 ID({rentId})를 기반으로 **모듈 장착 영상 URL 및 메타데이터**를 반환합니다.
 - **AWS S3 또는 파일 스토리지 서비스에서 저장된 영상**을 불러옵니다.

2. Request

요청 형식

Authorization: Bearer <token>

필드	타입	필수 여부	설명
rentId	String	✅ 필수	특정 대여 ID

3. Response

✅ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Module video retrieved successfully",
  "data": {
    "videoId": "VID123456",
    "rentId": "RENT001",
    "videoUrl": "https://s3.amazonaws.com/bucket-name/videos/RENT001-module.mp4",
    "videoMetadata": {
      "size": "25MB",
      "format": "mp4",
      "duration": "00:03:45",
      "recordedAt": "2025-01-12T14:30:00Z"
    }
  }
}
```

❌ 실패 시

```
{
  "resultCode": "FAILURE",
  "message": "Failed to retrieve module video",
  "errors": [
    {
      "field": "rentId",
      "message": "Invalid rent ID"
    },
    {
      "field": "rentId",
      "message": "No module video found for the given rent ID"
    }
  ]
}
```

4. 설명

✓ Request 설명

1. `rentId` (필수)
 - 특정 대여 ID(`rentId`)에 대한 **모듈 장착 영상**을 조회하는 API.
 - 경로 파라미터로 전달해야 함.

✓ Response 설명

1. 성공 (`resultCode = "SUCCESS"`)
 - `videoId`: 저장된 영상의 고유 ID.
 - `rentId`: 해당 영상이 속한 대여 ID.
 - `videoUrl`: 저장된 영상의 URL.
 - `videoMetadata`:
 - `size`: 영상 파일 크기 (예: `25MB`).
 - `format`: 영상 포맷 (예: `mp4`).
 - `duration`: 영상 길이 (예: `00:03:45`).
 - `recordedAt`: 촬영된 날짜 및 시간.
2. 실패 (`resultCode = "FAILURE"`)
 - `rentId`가 잘못되었거나 존재하지 않을 경우 `"Invalid rent ID"`
 - 해당 대여에 모듈 영상이 없는 경우 `"No module video found"`

5. 예시

✓ 성공 요청/응답

Request

```
Authorization: Bearer <token>
```

Response

```
{
  "resultCode": "SUCCESS",
  "message": "Module video retrieved successfully",
  "data": {
    "videoId": "VID123456",
    "rentId": "RENT001",
    "videoUrl": "https://s3.amazonaws.com/bucket-name/videos/RENT001-module.mp4",
    "videoMetadata": {
      "size": "25MB",
      "format": "mp4",
      "duration": "00:03:45",
      "recordedAt": "2025-01-12T14:30:00Z"
    }
  }
}
```

❌ 실패 요청/응답

Request (잘못된 **rentId**)

```
Authorization: Bearer <token>
```

Response

```
{
  "resultCode": "FAILURE",
  "message": "Failed to retrieve module video",
  "errors": [
    {
      "field": "rentId",
      "message": "Invalid rent ID"
    }
  ]
}
```

Request (영상이 없는 경우)

```
Authorization: Bearer <token>
```

Response

```
{
  "resultCode": "FAILURE",
  "message": "Failed to retrieve module video",
  "errors": [
    {
      "field": "rentId",
      "message": "No module video found for the given rent ID"
    }
  ]
}
```

정비 기록 조회

≡ API	/admin/maintenance/list
🕒 메소드	GET
≡ 보안	Bearer Token
☀ 상태	완료
≡ 설명	유지보수(정비) 기록을 조회합니다. 차량, 모듈, 옵션을 대상으로 필터링할 수 있습니다.

1. API 개요

- API 경로: /admin/maintenance/list
- 메서드: GET
- 설명: 유지보수(정비) 기록을 조회합니다. 차량, 모듈, 옵션을 대상으로 필터링할 수 있습니다.
- 인증 필요: Bearer Token 을 사용하여 인증.

2. Request

요청 필드

필드	타입	필수 여부	설명
type	String	선택	정비 대상 종류: vehicle , module , option
targetId	String	선택	특정 정비 대상 ID로 필터링
status	String	선택	정비 상태 필터링 (예: pending , in-progress , completed)
startDate	String	선택	조회 시작 날짜 (YYYY-MM-DD)
endDate	String	선택	조회 종료 날짜 (YYYY-MM-DD)
page	Integer	선택	페이지 번호 (기본값: 1)
pageSize	Integer	선택	페이지 크기 (기본값: 10)

요청 형식

```
{
  "type": "module",
  "targetId": "MOD001",
  "status": "pending",
  "startDate": "2025-01-01",
  "endDate": "2025-01-31",
  "page": 1,
  "pageSize": 5
}
```

3. Response

✅ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Maintenance records retrieved successfully",
  "data": {
```

```

    "maintenanceRecords": [
      {
        "maintenanceId": "MT001",
        "type": "module",
        "targetId": "MOD001",
        "description": "Battery replacement required",
        "status": "pending",
        "maintenanceDate": "2025-01-10",
        "cost": 150.00
      },
      {
        "maintenanceId": "MT002",
        "type": "module",
        "targetId": "MOD001",
        "description": "Firmware update required",
        "status": "in-progress",
        "maintenanceDate": "2025-01-15",
        "cost": 50.00
      }
    ],
    "pagination": {
      "currentPage": 1,
      "totalPages": 1,
      "totalItems": 2,
      "pageSize": 5
    }
  }
}

```

❌ 실패 시

```

{
  "resultCode": "FAILURE",
  "message": "Failed to retrieve maintenance records",
  "errors": [
    {
      "field": "type",
      "message": "Invalid type value. Must be 'vehicle', 'module', or 'option'."
    }
  ]
}

```

4. 설명

✅ Request 설명

1. **type** :
 - 정비 대상 종류 (**vehicle** , **module** , **option**)
 - 지정하지 않으면 모든 정비 기록을 반환.
2. **targetId** :
 - 특정 정비 대상 ID로 필터링 가능.
3. **status** :

- 정비 상태 필터링 (`pending` , `in-progress` , `completed`).
4. `startDate` , `endDate` :
 - 특정 기간 동안의 정비 기록을 조회.
 5. `page` :
 - 결과 페이지 번호 (기본값 `1`).
 6. `pageSize` :
 - 한 페이지당 반환할 항목 수 (기본값 `10`).

✅ Response 설명

1. `maintenanceRecords` :
 - 정비 기록 리스트.
 - 각 항목은 정비 ID, 대상, 설명, 상태, 정비 날짜, 비용 정보를 포함.
2. `pagination` :
 - 현재 페이지, 총 페이지 수, 총 항목 수, 페이지 크기 포함.

5. 예시

✅ 성공 요청/응답

Request

```
Authorization: Bearer <token>
```

Response

```
{
  "resultCode": "SUCCESS",
  "message": "Maintenance records retrieved successfully",
  "data": {
    "maintenanceRecords": [
      {
        "maintenanceId": "MT001",
        "type": "module",
        "targetId": "MOD001",
        "description": "Battery replacement required",
        "status": "pending",
        "maintenanceDate": "2025-01-10",
        "cost": 150.00
      }
    ],
    "pagination": {
      "currentPage": 1,
      "totalPages": 1,
      "totalItems": 1,
      "pageSize": 5
    }
  }
}
```

❌ 실패 요청/응답

Request (잘못된 `type` 값)

```
bash
복사편집
GET /admin/maintenance?type=invalidType HTTP/1.1
Authorization: Bearer <token>
```

Response

```
{
  "resultCode": "FAILURE",
  "message": "Failed to retrieve maintenance records",
  "errors": [
    {
      "field": "type",
      "message": "Invalid type value. Must be 'vehicle', 'module', or 'option'."
    }
  ]
}
```

✅ DB 반영

정비 기록 등록

≡ API	/admin/maintenance/register
🕒 메소드	POST
≡ 보안	Bearer Token
☀ 상태	완료
≡ 설명	유지보수(정비) 정보를 등록합니다. 차량, 모듈, 옵션에 대한 정비 요청을 생성할 수 있습니다

1. API 개요

- API 경로: /admin/maintenance/register
- 메서드: POST
- 설명: 유지보수(정비) 정보를 등록합니다. 차량, 모듈, 옵션에 대한 정비 요청을 생성할 수 있습니다.
- 인증 필요: Bearer Token 을 사용하여 인증.

2. Request

요청 필드

필드	타입	필수 여부	설명
type	String	✔ 필수	정비 대상 종류: vehicle , module , option
targetId	String	✔ 필수	정비 대상 ID (차량 ID, 모듈 ID, 옵션 ID)
issue	String	✔ 필수	정비 요청 사유 (고장 내용)
maintenanceDate	String	✔ 필수	정비 예정일 (YYYY-MM-DD)
cost	Decimal	✖ 선택	예상 정비 비용 (기본값 0.00)
status	String	✖ 선택	정비 상태 (pending 기본값)

요청 형식

```
{
  "type": "module",
  "targetId": "MOD001",
  "issue": "Battery replacement required",
  "maintenanceDate": "2025-02-15",
  "cost": 150.00,
  "status": "pending"
}
```

3. Response

✔ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Maintenance record registered successfully",
  "data": {
    "maintenanceId": "MT001",
    "type": "module",
    "targetId": "MOD001",
  }
}
```

```

    "issue": "Battery replacement required",
    "status": "pending",
    "maintenanceDate": "2025-02-15",
    "cost": 150.00,
    "created_at": "2025-02-10T12:00:00Z"
  }
}

```

❌ 실패 시

```

{
  "resultCode": "FAILURE",
  "message": "Failed to register maintenance record",
  "errors": [
    {
      "field": "type",
      "message": "Invalid type value. Must be 'vehicle', 'module', or 'option'."
    },
    {
      "field": "maintenanceDate",
      "message": "Invalid date format. Use YYYY-MM-DD."
    }
  ]
}

```

4. 설명

✅ Request 설명

1. **type**:
 - `vehicle`, `module`, `option` 중 하나 필수 선택.
2. **targetId**:
 - 정비 요청 대상의 ID.
3. **issue**:
 - 정비 요청 사유 (고장 내용).
4. **maintenanceDate**:
 - 정비 예정일 (`YYYY-MM-DD` 형식).
5. **cost**:
 - 예상 정비 비용 (`0.00` 기본값).
6. **status**:
 - `pending` (기본값), `in-progress`, `completed` 중 하나.

✅ Response 설명

1. **maintenanceId**:
 - 생성된 유지보수 ID.
2. **issue**:
 - 정비 요청 사유.
3. **status**:

- `pending`, `in-progress`, `completed` 상태 중 하나.

4. `created_at`:

- 정비 요청 등록 시각.

5. 예시

✅ 성공 요청/응답

Request

```
{
  "type": "module",
  "targetId": "MOD001",
  "issue": "Battery replacement required",
  "maintenanceDate": "2025-02-15",
  "cost": 150.00,
  "status": "pending"
}
```

Response

```
{
  "resultCode": "SUCCESS",
  "message": "Maintenance record registered successfully",
  "data": {
    "maintenanceId": "MT001",
    "type": "module",
    "targetId": "MOD001",
    "issue": "Battery replacement required",
    "status": "pending",
    "maintenanceDate": "2025-02-15",
    "cost": 150.00,
    "created_at": "2025-02-10T12:00:00Z"
  }
}
```

❌ 실패 요청/응답

Request (잘못된 `type` 값)

```
{
  "type": "invalidType",
  "targetId": "MOD001",
  "issue": "Battery replacement required",
  "maintenanceDate": "2025-02-15",
  "cost": 150.00,
  "status": "pending"
}
```

Response

```
{
  "resultCode": "FAILURE",
  "message": "Failed to register maintenance record",
  "errors": [
    {
      "field": "type",
      "message": "Invalid type value. Must be 'vehicle', 'module', or 'option'."
    }
  ]
}
```

정비 기록 수정

≡ API	/admin/maintenance/update/{maintenanceId}
⌵ 메소드	PUT
≡ 보안	Bearer Token
⚙ 상태	완료
≡ 설명	특정 유지보수(정비) 기록을 수정합니다. 정비 상태(<code>status</code>), 설명(<code>issue</code>), 정비 비용(<code>cost</code>) 등을 업데이트할 수 있습니다.

1. API 개요

- **API 경로:** `/admin/maintenance/update/{maintenanceId}`
- **메서드:** `PUT`
- **설명:** 특정 유지보수(정비) 기록을 수정합니다. 정비 상태(`status`), 설명(`issue`), 정비 비용(`cost`) 등을 업데이트할 수 있습니다.
- **인증 필요:** `Bearer Token` 을 사용하여 인증.

2. Request

요청 형식

```
{
  "status": "String",    // 정비 상태 (선택, 예: "pending", "in-progress", "completed")
  "issue": "String",     // 정비 설명 (선택)
  "cost": "Decimal"      // 정비 비용 (선택)
}
```

요청 필드 설명

필드	타입	필수 여부	설명
<code>status</code>	String	선택	정비 상태 (<code>pending</code> , <code>in-progress</code> , <code>completed</code>)
<code>issue</code>	String	선택	정비 설명 (고장 원인 및 해결 방법)
<code>cost</code>	Decimal	선택	정비 비용

3. Response

✅ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Maintenance record updated successfully",
  "data": {
    "maintenanceId": "MT001",
    "type": "module",
    "targetId": "MOD001",
    "issue": "Battery replaced successfully",
    "status": "completed",
    "cost": 150.00,
    "updatedAt": "2025-02-12T14:30:00Z"
  }
}
```

```
}  
}
```

❌ 실패 시

```
{  
  "resultCode": "FAILURE",  
  "message": "Failed to update maintenance record",  
  "errors": [  
    {  
      "field": "status",  
      "message": "Invalid status value. Must be 'pending', 'in-progress', or 'completed'."  
    },  
    {  
      "field": "maintenanceId",  
      "message": "Maintenance record not found"  
    }  
  ]  
}
```

4. 설명

✅ Request 설명

1. **status** :
 - 유지보수 상태를 업데이트합니다.
 - 유효 값: `pending`, `in-progress`, `completed`.
2. **issue** :
 - 정비 원인 및 해결 방법을 업데이트합니다.
3. **cost** :
 - 정비 비용을 업데이트합니다.

✅ Response 설명

1. **maintenanceId** :
 - 수정된 유지보수 기록의 ID.
2. **type** :
 - 유지보수 대상 종류 (예: `vehicle`, `module`, `option`).
3. **targetId** :
 - 유지보수 대상의 ID.
4. **issue** :
 - 업데이트된 유지보수 설명.
5. **status** :
 - 업데이트된 유지보수 상태.
6. **cost** :
 - 업데이트된 정비 비용.

7. `updatedAt`:

- 수정된 시간.

5. 예시

✅ 성공 요청/응답

Request

```
{
  "status": "completed",
  "issue": "Battery replaced successfully",
  "cost": 150.00
}
```

Response

```
{
  "resultCode": "SUCCESS",
  "message": "Maintenance record updated successfully",
  "data": {
    "maintenanceId": "MT001",
    "type": "module",
    "targetId": "MOD001",
    "issue": "Battery replaced successfully",
    "status": "completed",
    "cost": 150.00,
    "updatedAt": "2025-02-12T14:30:00Z"
  }
}
```

❌ 실패 요청/응답

Request (잘못된 `status` 값)

```
{
  "status": "invalidStatus",
  "issue": "Attempted update with invalid status",
  "cost": -50
}
```

Response

```
{
  "resultCode": "FAILURE",
  "message": "Failed to update maintenance record",
  "errors": [
    {
      "field": "status",
      "message": "Invalid status value. Must be 'pending', 'in-progress', or 'complete d'."
    },
  ],
  {

```

```
    "field": "maintenanceId",  
    "message": "Maintenance record not found"  
  },  
  {  
    "field": "cost",  
    "message": "Cost must be a positive value."  
  }  
]  
}
```


정비 기록 삭제

≡ API	/admin/maintenance/delete/{maintenanceId}
🕒 메소드	DELETE
≡ 보안	Bearer Token
☀ 상태	완료
≡ 설명	특정 유지보수(정비) 기록을 삭제합니다.

1. API 개요

- **API 경로:** /admin/maintenance/delete/{maintenanceId}
- **메서드:** DELETE
- **설명:** 특정 유지보수(정비) 기록을 삭제합니다.
- **인증 필요:** Bearer Token 을 사용하여 인증.

2. Request

요청 형식

Headers

```
Authorization: Bearer <token>
```

요청 URL 예시

```
DELETE /admin/maintenance/delete/MT001
```

- `maintenanceId` : 삭제할 유지보수 기록의 고유 ID.

3. Response

✅ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Maintenance record deleted successfully",
  "data": {
    "maintenanceId": "MT001"
  }
}
```

❌ 실패 시

```
{
  "resultCode": "FAILURE",
  "message": "Failed to delete maintenance record",
  "errors": [
    {
      "field": "maintenanceId",
      "message": "Maintenance record not found"
    }
  ]
}
```

```

    },
    {
      "field": "maintenanceId",
      "message": "Cannot delete an active maintenance record"
    }
  ]
}

```

4. 설명

✅ Request 설명

1. `maintenanceId` :
 - 삭제하려는 유지보수 기록의 고유 ID를 URL 경로로 전달합니다.
 - 유지보수 기록이 존재하지 않으면 삭제할 수 없습니다.

✅ Response 설명

1. `maintenanceId` :
 - 삭제된 유지보수 기록의 ID.
2. `resultCode` :
 - 요청 성공 여부 (`SUCCESS` 또는 `FAILURE`).
3. `message` :
 - 요청 성공 또는 실패 메시지.
4. `errors` :
 - 실패 시, 발생한 오류의 필드와 메시지.

5. 예시

✅ 성공 요청/응답

Request

```

DELETE /admin/maintenance/delete/MT001
Authorization: Bearer <token>

```

Response

```

{
  "resultCode": "SUCCESS",
  "message": "Maintenance record deleted successfully",
  "data": {
    "maintenanceId": "MT001"
  }
}

```

❌ 실패 요청/응답

Request (잘못된 유지보수 ID)

```
DELETE /admin/maintenance/delete/INVALID_MT_ID
Authorization: Bearer <token>
```

Response

```
{
  "resultCode": "FAILURE",
  "message": "Failed to delete maintenance record",
  "errors": [
    {
      "field": "maintenanceId",
      "message": "Maintenance record not found"
    }
  ]
}
```

Request (진행 중인 유지보수 삭제 시도)

```
DELETE /admin/maintenance/delete/MT002
Authorization: Bearer <token>
```

Response

```
{
  "resultCode": "FAILURE",
  "message": "Failed to delete maintenance record",
  "errors": [
    {
      "field": "maintenanceId",
      "message": "Cannot delete an active maintenance record"
    }
  ]
}
```

대여 번호 배정

≡ API	/ros2/vehicle/assign/{vehicleId}
🕒 메소드	Service
☀ 상태	완료
≡ 설명	서버가 WebSocket을 통해 ROS2에 차량 대기 배정을 요청.

1. 개요

- 서비스명: /ros2/vehicle/assign/{vehicleId}
- 통신 방식: WebSocket (ROS2 Service)
- 요청 방향: 서버 → ROS2
- 응답 방향: ROS2 → 서버
- 설명:
 - 서버가 WebSocket을 통해 ROS2에 차량 대기 배정을 요청.
 - ROS2는 차량 상태를 "대기 중"(**waiting**)으로 변경한 후, 응답을 반환.

2. WebSocket 요청 (서버 → ROS2)

서비스 호출 형식

WebSocket Service: /ros2/vehicle/assign/{vehicleId}

요청 형식 (JSON)

```
{
  "rentId": "String"    // 배정할 대여 번호 (필수)
}
```

요청 필드 설명

필드	타입	필수 여부	설명
vehicleId	String	✓ 필수	WebSocket 경로에 포함된 차량 ID (DB에서 "사용 중"이 아닌 차량)
rentId	String	✓ 필수	대여 고유 번호

3. WebSocket 응답 (ROS2 → 서버)

✓ 배정 성공

```
{
  "resultCode": "SUCCESS",
  "message": "Rent ID assigned successfully",
  "data": {
    "vehicleId": "VEH001",    // 배정된 차량 ID
    "rentId": "RENT001"      // 배정된 대여 번호
  }
}
```

✖ 배정 실패 (차량 없음 또는 배정 불가능)

```
{
  "resultCode": "FAILURE",
  "message": "Failed to assign Rent ID",
  "errors": [
    {
      "field": "vehicleId",
      "message": "Vehicle not found or unavailable"
    }
  ]
}
```

4. ROS2 WebSocket 서비스 처리 흐름

단계	ROS2 처리 내용
1	서버가 WebSocket으로 <code>/ros2/vehicle/assign/{vehicleId}</code> 호출
2	ROS2가 차량의 현재 상태를 조회
3	차량이 "사용 중(active)"이 아닌 경우 상태를 "대기 중(waiting)"으로 변경
4	변경 성공 시, WebSocket을 통해 서버에 성공 응답 전송
5	변경 실패 시, WebSocket을 통해 서버에 실패 응답 전송

5. ROS2 내부 로직

◆ 차량 상태 조회

```
vehicle_status = check_vehicle_status(vehicle_id)
if vehicle_status == "active":
    return {"resultCode": "FAILURE", "message": "Vehicle is currently in use"}
```

- 현재 차량 상태가 "사용 중(active)"이면 배정 불가능.

◆ 차량 상태 "대기 중"으로 변경

```
update_vehicle_status(vehicle_id, "waiting")
return {"resultCode": "SUCCESS", "message": "Rent ID assigned successfully", "data":
{"vehicleId": vehicle_id, "rentId": rent_id}}
```

- 차량 상태를 "waiting"으로 변경 후 성공 응답 반환.

◆ 실패 처리 (차량 없음)

```
return {"resultCode": "FAILURE", "message": "Vehicle not found or unavailable"}
```

- 차량이 존재하지 않거나 사용 불가능한 경우, 실패 응답 반환.

6. WebSocket 통신 흐름 (Server ↔ ROS2)

1 서버 → ROS2 요청

```
[WebSocket 요청] /ros2/vehicle/assign/VEH001
```

```
{
  "rentId": "RENT001"
}
```

2 ROS2 차량 상태 확인

- 차량이 "사용 중(active)"이면 실패 반환.
- "대기 중(waiting)"으로 변경 후 응답.

3 ROS2 → 서버 응답

✓ 배정 성공

```
{
  "resultCode": "SUCCESS",
  "message": "Rent ID assigned successfully",
  "data": {
    "vehicleId": "VEH001",
    "rentId": "RENT001"
  }
}
```

✗ 배정 실패 (차량 없음 또는 사용 불가)

```
{
  "resultCode": "FAILURE",
  "message": "Failed to assign Rent ID",
  "errors": [
    {
      "field": "vehicleId",
      "message": "Vehicle not found or unavailable"
    }
  ]
}
```

모듈 장착 영상 저장

≡ API	/ros2/moduleVideo
🕒 메소드	Service
☀ 상태	완료
≡ 설명	ROS2가 차량이 장착한 모듈 정보를 감지하면 이를 서버에 전송.

1. 개요

- 서비스명: /ros2/moduleVideo
- 통신 방식: WebSocket (ROS2 → Server)
- 요청 방향: ROS2 → Server
- 응답 방향: Server → ROS2
- 설명:
 - ROS2가 차량이 장착한 모듈 정보를 감지하면 이를 서버에 전송.
 - 서버는 모듈 장착 영상을 저장하고, 응답을 반환.

2. WebSocket 요청 (ROS2 → Server)

서비스 호출 형식

WebSocket Service: /ros2/moduleVideo

요청 형식 (JSON)

```
{
  "vehicleId": "String",      // 차량 ID (필수)
  "rentId": "String",        // 대여 고유 ID (필수)
  "moduleName": "String",    // 인식한 모듈 이름 (필수)
  "videoData": "base64-encoded-video-data" // 모듈 장착 영상 데이터 (필수)
}
```

요청 필드 설명

필드	타입	필수 여부	설명
vehicleId	String	✔ 필수	차량의 고유 ID
rentId	String	✔ 필수	대여 고유 번호
moduleName	String	✔ 필수	장착된 모듈 이름
videoData	String (Base64)	✔ 필수	모듈 장착 영상 데이터

3. WebSocket 응답 (Server → ROS2)

✔ 영상 저장 성공

```
{
  "resultCode": "SUCCESS",
}
```

```

    "message": "Module video saved successfully"
  }

```

❌ 영상 저장 실패

```

{
  "resultCode": "FAILURE",
  "message": "Failed to save module video",
  "errors": [
    {
      "field": "videoData",
      "message": "Invalid video format"
    }
  ]
}

```

4. ROS2 WebSocket 서비스 처리 흐름

단계	ROS2 처리 내용
1	차량이 모듈을 인식하고 장착
2	모듈 장착 영상을 촬영
3	촬영된 영상을 Base64로 인코딩
4	WebSocket을 통해 서버에 전송
5	서버가 영상을 저장 후 응답 반환

5. ROS2 내부 로직

◆ 모듈 인식 및 영상 촬영

```

module_name = detect_module()
video_data = record_video()
base64_video = encode_video_to_base64(video_data)

```

- ROS2가 차량이 감지한 모듈을 인식.
- 해당 모듈 장착 과정을 영상으로 촬영.
- 촬영된 영상을 Base64로 변환.

◆ WebSocket을 통해 서버에 영상 전송

```

data = {
  "vehicleId": "VEH001",
  "rentId": "RENT001",
  "moduleName": "Battery Module",
  "videoData": base64_video
}

send_websocket_request("/ros2/moduleVideo", data)

```

- 차량 ID(`vehicleId`) 및 대여 ID(`rentId`)를 포함.
- 촬영된 영상 데이터를 서버로 전송.

◆ 서버 응답 처리

```
response = receive_websocket_response()

if response["resultCode"] == "SUCCESS":
    print("✅ 영상 저장 성공")
else:
    print(f"❌ 저장 실패: {response['message']}")
```

- 서버 응답을 확인하여 성공 여부 처리.

6. WebSocket 통신 흐름 (ROS2 ↔ Server)

1 ROS2 → 서버 요청

[WebSocket 요청] /ros2/moduleVideo

```
{
  "vehicleId": "VEH001",
  "rentId": "RENT001",
  "moduleName": "Battery Module",
  "videoData": "base64-encoded-video-data"
}
```

2 서버가 영상 저장 후 응답

✅ 저장 성공

```
{
  "resultCode": "SUCCESS",
  "message": "Module video saved successfully"
}
```

❌ 저장 실패 (잘못된 데이터)

```
{
  "resultCode": "FAILURE",
  "message": "Failed to save module video",
  "errors": [
    {
      "field": "videoData",
      "message": "Invalid video format"
    }
  ]
}
```

차량 데이터 확인

≡ API	/ros2/vehicle/status
🔍 메소드	Topic
⚙ 상태	완료
≡ 설명	자율주행 차량이 실시간으로 상태 데이터를 서버에 퍼블리시

🚗 ROS2 차량 운행 상태 퍼블리시 토픽 (Publish)

- 토픽명: /ros2/vehicle/status
- 통신 방식: WebSocket (ROS2 → Server)
- 메시지 유형: JSON
- 설명:
 - 자율주행 차량이 실시간으로 상태 데이터를 서버에 퍼블리시.
 - DB 및 API와 용어를 통일하여 정리.
 - SLAM 기반 좌표 및 차량 상태 정보를 포함.

1. 메시지 포맷 (ROS2 → Server)

```
{
  "vehicleId": 101,           // 차량 ID (DB: vehicle.vehicle_id)
  "rentId": 1001,            // 대여 ID (DB: rent_history.rent_id)
  "isArrived": false,        // 목적지 도착 여부
  "currentLocation": {       // 현재 차량 위치 정보
    "x": 37.7749,
    "y": -122.4194
  },
  "autonomousArrivalPoint": { // SLAM 기반 목적지 좌표
    "x": 40.1111,
    "y": -100.4194
  },
  "estimatedArrivalTime": "2025-01-13T14:30:00Z", // 예상 도착 시간 (ETA)
  "totalDistance": 120,      // 총 주행 거리
  "plannedPath": [           // SLAM 기반 예상 경로
    { "x": 37.7750, "y": -122.4195 },
    { "x": 37.7760, "y": -122.4185 }
  ],
  "slamMapData": "base64-encoded-map-data", // SLAM 맵 데이터 (이미지 또는 맵 포맷)
  "status": {                // 차량 및 모듈 상태
    "vehicleStatus": {
      "batteryLevel": 85,     // 차량 배터리 상태 (%)
      "lightIntensity": 80    // 차량 조명 밝기 (%)
    },
    "module": {               // 장착된 모듈 정보
      "moduleId": 201,
      "moduleType": "캠핑카 모듈",
      "status": "active"
    }
  },
  "installedOptions": [      // 장착된 옵션 상태
```

```

{
  "optionId": 301,
  "optionName": "물탱크",
  "status": "잔여량: 50L"
},
{
  "optionId": 302,
  "optionName": "배터리",
  "status": "잔여량: 80%"
}
]
}
}

```

2. 필드 설명 (DB 및 기존 API 반영)

필드	타입	DB 매핑	설명
<code>vehicleId</code>	Int	<code>vehicle.vehicle_id</code>	차량 ID
<code>rentId</code>	Int	<code>rent_history.rent_id</code>	대여 ID
<code>isArrived</code>	Boolean	-	차량이 목적지에 도착했는지 여부
<code>currentLocation</code>	Object	<code>vehicle.departure_point</code>	차량의 현재 GPS 좌표
<code>autonomousArrivalPoint</code>	Object	<code>rent_history.autonomous_arrival_point</code>	자율주행 목적지 GPS 좌표
<code>estimatedArrivalTime</code>	String (ISO 8601)	-	예상 도착 시간 (ETA)
<code>totalDistance</code>	Int	<code>rent_history.total_distance</code>	차량의 총 이동 거리
<code>plannedPath</code>	Array	-	SLAM 데이터 기반 예상 경로
<code>slamMapData</code>	String (Base64)	-	SLAM 맵 데이터 (이미지 또는 맵 포맷)
<code>status.vehicleStatus.batteryLevel</code>	Int (%)	-	차량 배터리 잔량
<code>status.vehicleStatus.lightIntensity</code>	Int (%)	-	차량 조명 밝기
<code>status.module.moduleId</code>	Int	<code>module.module_id</code>	장착된 모듈 ID
<code>status.module.moduleType</code>	String	<code>module.module_type</code>	장착된 모듈 타입
<code>status.module.status</code>	String	<code>module.status</code>	모듈 상태 (<code>active</code> , <code>maintenance</code> , <code>inactive</code>)
<code>status.installedOptions[].optionId</code>	Int	<code>option.option_id</code>	장착된 옵션 ID
<code>status.installedOptions[].optionName</code>	String	<code>option.option_name</code>	옵션 이름
<code>status.installedOptions[].status</code>	String	<code>option_usage_history.status</code>	옵션 상태 정보

3. ROS2 → Server 메시지 흐름

◆ 차량 상태 지속 전송 (Publish)

1. ROS2가 차량 위치 및 상태를 감지.
2. WebSocket을 통해 `/ros2/vehicle/status` 토픽으로 지속적인 퍼블리싱.
3. 서버가 데이터를 받아 실시간 모니터링 및 로그 저장.

4. ROS2 내부 구현 예제

◆ 차량 상태 지속 전송 (WebSocket Publish)

```
import json
import websocket
import time

def send_vehicle_status():
    ws = websocket.WebSocket()
    ws.connect("ws://server-address:port/ros2/vehicle/status")

    while True:
        message = {
            "vehicleId": 101,
            "rentId": 1001,
            "isArrived": False,
            "currentLocation": {
                "x": 37.7749,
                "y": -122.4194
            },
            "autonomousArrivalPoint": {
                "x": 40.1111,
                "y": -100.4194
            },
            "estimatedArrivalTime": "2025-01-13T14:30:00Z",
            "totalDistance": 120,
            "plannedPath": [
                {"x": 37.7750, "y": -122.4195},
                {"x": 37.7760, "y": -122.4185}
            ],
            "slamMapData": "base64-encoded-map-data",
            "status": {
                "vehicleStatus": {
                    "batteryLevel": 85,
                    "lightIntensity": 80
                },
                "module": {
                    "moduleId": 201,
                    "moduleType": "캠핑카 모듈",
                    "status": "active"
                },
                "installedOptions": [
                    {"optionId": 301, "optionName": "물탱크", "status": "잔여량: 50L"},
                    {"optionId": 302, "optionName": "배터리", "status": "잔여량: 80%"}
                ]
            }
        }

        ws.send(json.dumps(message))
        time.sleep(5) # 5초마다 업데이트 전송

send_vehicle_status()
```

5. ROS2 웹소켓 통신 흐름

1 ROS2 → 서버 (차량 상태 전송)

[WebSocket 메시지] /ros2/vehicle/status

```
{
  "vehicleId": 101,
  "rentId": 1001,
  "isArrived": false,
  "currentLocation": {
    "x": 37.7749,
    "y": -122.4194
  },
  "autonomousArrivalPoint": {
    "x": 40.1111,
    "y": -100.4194
  },
  "estimatedArrivalTime": "2025-01-13T14:30:00Z",
  "totalDistance": 120,
  "plannedPath": [
    {"x": 37.7750, "y": -122.4195},
    {"x": 37.7760, "y": -122.4185}
  ],
  "slamMapData": "base64-encoded-map-data",
  "status": {
    "vehicleStatus": {
      "batteryLevel": 85,
      "lightIntensity": 80
    },
    "module": {
      "moduleId": 201,
      "moduleType": "캠핑카 모듈",
      "status": "active"
    },
    "installedOptions": [
      {"optionId": 301, "optionName": "물탱크", "status": "잔여량: 50L"},
      {"optionId": 302, "optionName": "배터리", "status": "잔여량: 80%"}
    ]
  }
}
```

자율 주행 영상 저장

≡ API	/ros2/vehicle/autonomous_video
🕒 메소드	Service
☀ 상태	완료
≡ 설명	차량이 반납되거나 대여가 취소될 때 자율주행 중 촬영한 영상 데이터를 서버에 저장.

- 통신 방식: WebSocket (ROS2 → Server)
- 토픽명: /ros2/vehicle/autonomous_video
- 설명:
 - 차량이 반납되거나 대여가 취소될 때 자율주행 중 촬영한 영상 데이터를 서버에 저장.
 - WebSocket을 이용한 ROS2 → Server 통신.
 - Base64 인코딩된 영상 데이터를 포함.

1. 메시지 포맷 (ROS2 → Server)

```
{
  "vehicleId": 101,          // 차량 ID
  "rentId": 1001,           // 대여 ID
  "eventType": "return",    // 이벤트 유형: "return" (반납), "cancel" (취소)
  "videoData": "base64-encoded-video-data", // 자율주행 중 촬영한 영상 데이터
  "recordedAt": "2025-01-15T18:30:00Z"      // 영상 촬영 완료 시간 (ISO 8601 형식)
}
```

2. 필드 설명 (DB 및 기존 API 반영)

필드	타입	DB 매핑	설명
vehicleId	Int	vehicle.vehicle_id	차량 ID
rentId	Int	rent_history.rent_id	대여 ID
eventType	String	-	반납(return) 또는 취소(cancel)
videoData	String (Base64)	video_storage.video_url	촬영된 자율주행 영상 데이터
recordedAt	String (ISO 8601)	video_storage.recorded_at	영상 촬영 완료 시간

3. ROS2 → Server 메시지 흐름

◆ 차량 반납 또는 취소 시 영상 데이터 전송

1. ROS2가 반납/취소 이벤트 감지
2. 자율주행 중 촬영된 영상 데이터를 WebSocket을 통해 서버로 전송
3. 서버가 데이터를 저장하고 DB에 기록

4. Response (Server → ROS2)

✅ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Autonomous driving video saved successfully",
  "data": {
    "videoId": 301,          // 저장된 영상의 고유 ID
    "vehicleId": 101,       // 차량 ID
    "rentId": 1001,        // 대여 ID
    "eventType": "return",  // 반납 또는 취소
    "videoUrl": "https://storage.example.com/videos/301.mp4",
    "recordedAt": "2025-01-15T18:30:00Z"
  }
}
```

❌ 실패 시

```
{
  "resultCode": "FAILURE",
  "message": "Failed to save autonomous driving video",
  "errors": [
    {
      "field": "vehicleId",
      "message": "Invalid vehicle ID"
    }
  ]
}
```

5. ROS2 내부 구현 예제 (WebSocket 통신)

```
import json
import websocket

def send_autonomous_video(vehicle_id, rent_id, event_type, video_data, recorded_at):
    ws = websocket.WebSocket()
    ws.connect("ws://server-address:port/ros2/vehicle/autonomous_video")

    message = {
        "vehicleId": vehicle_id,
        "rentId": rent_id,
        "eventType": event_type,
        "videoData": video_data, # Base64 인코딩된 영상 데이터
        "recordedAt": recorded_at
    }

    ws.send(json.dumps(message))
    response = json.loads(ws.recv())
    print(response) # 서버 응답 출력
    ws.close()

# 테스트 실행
send_autonomous_video(
    vehicle_id=101,
    rent_id=1001,
    event_type="return",
```

```
video_data="base64-encoded-video-data",
recorded_at="2025-01-15T18:30:00Z"
)
```

6. ROS2 웹소켓 통신 흐름

1 ROS2 → 서버 (자율주행 영상 저장 요청)

[WebSocket 메시지] /ros2/vehicle/autonomous_video

```
{
  "vehicleId": 101,
  "rentId": 1001,
  "eventType": "return",
  "videoData": "base64-encoded-video-data",
  "recordedAt": "2025-01-15T18:30:00Z"
}
```

2 서버 → ROS2 응답 (영상 저장 결과)

✅ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Autonomous driving video saved successfully",
  "data": {
    "videoId": 301,
    "vehicleId": 101,
    "rentId": 1001,
    "eventType": "return",
    "videoUrl": "https://storage.example.com/videos/301.mp4",
    "recordedAt": "2025-01-15T18:30:00Z"
  }
}
```

❌ 실패 시

```
{
  "resultCode": "FAILURE",
  "message": "Failed to save autonomous driving video",
  "errors": [
    {
      "field": "vehicleId",
      "message": "Invalid vehicle ID"
    }
  ]
}
```


차량 대여 취소

≡ API	/ros2/vehicle/{vehicleId}/cancel_rent
🕒 메소드	Service
☀ 상태	완료
≡ 설명	사용자가 대여 취소를 요청하면, 서버는 ROS2에게 해당 차량을 "대기 상태"로 변경하도록 요청

🚗 차량 대여 취소 시 ROS2 서비스 호출 (Server → ROS2)

- 통신 방식: WebSocket (Server → ROS2)
- 서비스 경로: /ros2/vehicle/{vehicleId}/cancel_rent
- 설명:
 - 사용자가 대여 취소를 요청하면, 서버는 ROS2에게 해당 차량을 "대기 상태"로 변경하도록 요청.
 - ROS2는 해당 차량을 즉시 대기 상태(**waiting**)로 업데이트.

1. Request (Server → ROS2)

요청 형식

```
{
  "rentId": 1001  // 취소할 대여 ID (DB: rent_history.rent_id)
}
```

요청 필드 설명

필드	타입	설명
rentId	Int	취소할 대여 ID

2. Response (ROS2 → Server)

✅ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Rental cancellation processed successfully",
  "data": {
    "vehicleId": 101,  // 대여 취소된 차량 ID
    "status": "waiting"  // 업데이트된 차량 상태
  }
}
```

❌ 실패 시

```
{
  "resultCode": "FAILURE",
  "message": "Failed to process rental cancellation",
  "errors": [
    {
      "field": "rentId",
    }
  ]
}
```

```

        "message": "Rental ID not found or already completed"
    }
]
}

```

3. ROS2 서비스 네임스페이스 설정

- 서비스 경로: `/ros2/vehicle/{vehicleId}/cancel_rent`
- 설명: ROS2에서 해당 차량(`vehicleId`)에 대한 대여 취소를 처리.
- ROS2에서 수신하는 메시지:

```

{
  "rentId": 1001
}

```

- ROS2에서 응답하는 메시지:

```

{
  "resultCode": "SUCCESS",
  "message": "Rental cancellation processed successfully",
  "data": {
    "vehicleId": 101,
    "status": "waiting"
  }
}

```

4. Server → ROS2 메시지 흐름

- 1 사용자가 차량 대여 취소 요청 → `/user/rent/cancel/{rentId}`
- 2 서버가 해당 차량 ID(`vehicleId`) 조회 후 ROS2에 요청
- 3 서버가 ROS2에 취소 요청 WebSocket 메시지 전송 (`/ros2/vehicle/{vehicleId}/cancel_rent`)
- 4 ROS2가 차량을 `waiting` 상태로 업데이트
- 5 ROS2가 서버에 응답 (`SUCCESS` or `FAILURE`)
- 6 서버가 사용자에게 취소 완료 응답

5. ROS2 웹소켓 예제 (서버 → ROS2 서비스 호출)

```

import json
import websocket

def cancel_rental(vehicle_id, rent_id):
    ws = websocket.WebSocket()
    ws.connect(f"ws://ros2-server-address:port/ros2/vehicle/{vehicle_id}/cancel_rent")

    message = {
        "rentId": rent_id
    }

    ws.send(json.dumps(message))
    response = json.loads(ws.recv())
    print(response) # 서버 응답 출력

```

```
ws.close()

# 테스트 실행
cancel_rental(vehicle_id=101, rent_id=1001)
```

6. Server → ROS2 요청 및 응답 예제

✅ Server → ROS2 요청

```
POST /ros2/vehicle/101/cancel_rent
```

```
{
  "rentId": 1001
}
```

✅ ROS2 → Server 응답 (성공)

```
{
  "resultCode": "SUCCESS",
  "message": "Rental cancellation processed successfully",
  "data": {
    "vehicleId": 101,
    "status": "waiting"
  }
}
```

❌ ROS2 → Server 응답 (실패: 잘못된 대여 ID)

```
{
  "resultCode": "FAILURE",
  "message": "Failed to process rental cancellation",
  "errors": [
    {
      "field": "rentId",
      "message": "Rental ID not found or already completed"
    }
  ]
}
```

차량 대여 완료

≡ API	/ros2/vehicle/{vehicleId}/complete_rent
🕒 메소드	Service
☀ 상태	완료
≡ 설명	사용자가 대여 완료를 요청하면, 서버는 ROS2에게 해당 차량을 "대여 완료" (completed) 상태로 변경하도록 요청.

🚗 차량 대여 완료 시 ROS2 서비스 호출 (Server → ROS2)

- 통신 방식: WebSocket (Server → ROS2)
- 서비스 경로: /ros2/vehicle/{vehicleId}/complete_rent
- 설명:
 - 사용자가 대여 완료를 요청하면, 서버는 ROS2에게 해당 차량을 "대여 완료" (**completed**) 상태로 변경하도록 요청.
 - ROS2는 차량의 최종 상태를 확인하고, 차량을 "대기 상태" (**waiting**) 로 업데이트.

1. Request (Server → ROS2)

요청 형식

```
{
  "rentId": 1001  // 완료할 대여 ID (DB: rent_history.rent_id)
}
```

요청 필드 설명

필드	타입	설명
rentId	Int	완료할 대여 ID

2. Response (ROS2 → Server)

✅ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Rental completion processed successfully",
  "data": {
    "vehicleId": 101,  // 대여 완료된 차량 ID
    "status": "waiting" // 업데이트된 차량 상태
  }
}
```

❌ 실패 시

```
{
  "resultCode": "FAILURE",
  "message": "Failed to process rental completion",
  "errors": [
    {
      "field": "rentId",
    }
  ]
}
```

```

        "message": "Rental ID not found or already completed"
    }
]
}

```

3. ROS2 서비스 네임스페이스 설정

- 서비스 경로: `/ros2/vehicle/{vehicleId}/complete_rent`
- 설명: ROS2에서 해당 차량(`vehicleId`)에 대한 대여 완료 처리를 담당.
- ROS2에서 수신하는 메시지:

```

{
  "rentId": 1001
}

```

- ROS2에서 응답하는 메시지:

```

{
  "resultCode": "SUCCESS",
  "message": "Rental completion processed successfully",
  "data": {
    "vehicleId": 101,
    "status": "waiting"
  }
}

```

4. Server → ROS2 메시지 흐름

- 1 사용자가 차량 대여 완료 요청 → `/user/rent/complete/{rentId}`
- 2 서버가 해당 차량 ID(`vehicleId`) 조회 후 ROS2에 요청
- 3 서버가 ROS2에 대여 완료 WebSocket 메시지 전송 (`/ros2/vehicle/{vehicleId}/complete_rent`)
- 4 ROS2가 차량을 `waiting` 상태로 업데이트
- 5 ROS2가 서버에 응답 (`SUCCESS` or `FAILURE`)
- 6 서버가 사용자에게 대여 완료 응답

5. ROS2 웹소켓 예제 (서버 → ROS2 서비스 호출)

```

import json
import websocket

def complete_rental(vehicle_id, rent_id):
    ws = websocket.WebSocket()
    ws.connect(f'ws://{ros2_server_address:port}/ros2/vehicle/{vehicle_id}/complete_rent')

    message = {
        "rentId": rent_id
    }

    ws.send(json.dumps(message))
    response = json.loads(ws.recv())

```

```
print(response) # 서버 응답 출력
ws.close()

# 테스트 실행
complete_rental(vehicle_id=101, rent_id=1001)
```

6. Server → ROS2 요청 및 응답 예제

✅ Server → ROS2 요청

```
POST /ros2/vehicle/101/complete_rent
```

```
{
  "rentId": 1001
}
```

✅ ROS2 → Server 응답 (성공)

```
{
  "resultCode": "SUCCESS",
  "message": "Rental completion processed successfully",
  "data": {
    "vehicleId": 101,
    "status": "waiting"
  }
}
```

❌ ROS2 → Server 응답 (실패: 잘못된 대여 ID)

```
{
  "resultCode": "FAILURE",
  "message": "Failed to process rental completion",
  "errors": [
    {
      "field": "rentId",
      "message": "Rental ID not found or already completed"
    }
  ]
}
```

차량 목적지 설정

≡ API	/ros2/vehicle/{vehicleId}/setDestination
🕒 메소드	Service
☀ 상태	완료
≡ 설명	사용자가 운행 중 새로운 목적지 를 설정하면, 서버가 ROS2에게 해당 목적지를 전달.

🚗 운행 중 목적지 설정 서비스 (Server → ROS2)

- 통신 방식: WebSocket (Server → ROS2)
- 서비스 경로: /ros2/vehicle/{vehicleId}/setDestination
- 설명:
 - 사용자가 **운행 중 새로운 목적지**를 설정하면, 서버가 ROS2에게 해당 목적지를 전달.
 - ROS2는 **SLAM 데이터 기반**으로 경로를 계획하고 이동 경로를 업데이트.

1. Request (Server → ROS2)

요청 형식

```
{
  "rentId": 1001,           // 대여 ID
  "destination": {         // 목적지 좌표 (SLAM 기반)
    "x": 12.313,
    "y": 32.3232
  }
}
```

요청 필드 설명

필드	타입	설명
rentId	Int	현재 운행 중인 대여 ID
destination	Object	목적지 좌표 (SLAM x, y)
destination.x	Float	목적지 x 좌표
destination.y	Float	목적지 y 좌표

2. Response (ROS2 → Server)

✅ 성공 시

```
{
  "resultCode": "SUCCESS",
  "message": "Destination set successfully",
  "data": {
    "rentId": 1001,
    "destination": {
      "x": 12.313,
      "y": 32.3232
    }
  }
}
```

```
}  
}
```

❌ 실패 시

```
{  
  "resultCode": "FAILURE",  
  "message": "Failed to set destination",  
  "errors": [  
    {  
      "field": "destination",  
      "message": "Invalid destination coordinates"  
    },  
    {  
      "field": "rentId",  
      "message": "Rental ID not found or inactive"  
    }  
  ]  
}
```

3. ROS2 서비스 네임스페이스 설정

- 서비스 경로: `/ros2/vehicle/{vehicleId}/setDestination`
- 설명: ROS2에서 해당 차량(`vehicleId`)의 새로운 목적지를 설정.
- ROS2에서 수신하는 메시지:

```
{  
  "rentId": 1001,  
  "destination": {  
    "x": 12.313,  
    "y": 32.3232  
  }  
}
```

- ROS2에서 응답하는 메시지:

```
{  
  "resultCode": "SUCCESS",  
  "message": "Destination set successfully",  
  "data": {  
    "rentId": 1001,  
    "destination": {  
      "x": 12.313,  
      "y": 32.3232  
    }  
  }  
}
```

4. Server → ROS2 메시지 흐름

- 1 사용자가 운행 중 목적지 변경 요청 → `/user/rent/{rentId}/updateDestination`
- 2 서버가 해당 차량(`vehicleId`)을 조회 후 ROS2에 요청

- 3 서버가 ROS2에 목적지 설정 WebSocket 메시지 전송 (/ros2/vehicle/{vehicleId}/setDestination)
- 4 ROS2가 SLAM 데이터 기반으로 경로 재설정
- 5 ROS2가 서버에 응답 (SUCCESS or FAILURE)
- 6 서버가 사용자에게 목적지 변경 완료 응답

5. ROS2 웹소켓 예제 (서버 → ROS2 서비스 호출)

```
import json
import websocket

def set_destination(vehicle_id, rent_id, x, y):
    ws = websocket.WebSocket()
    ws.connect(f"ws://ros2-server-address:port/ros2/vehicle/{vehicle_id}/setDestination")

    message = {
        "rentId": rent_id,
        "destination": {
            "x": x,
            "y": y
        }
    }

    ws.send(json.dumps(message))
    response = json.loads(ws.recv())
    print(response) # 서버 응답 출력
    ws.close()

# 테스트 실행
set_destination(vehicle_id=101, rent_id=1001, x=12.313, y=32.3232)
```

6. Server → ROS2 요청 및 응답 예제

✓ Server → ROS2 요청

```
POST /ros2/vehicle/101/setDestination
```

```
{
  "rentId": 1001,
  "destination": {
    "x": 12.313,
    "y": 32.3232
  }
}
```

✓ ROS2 → Server 응답 (성공)

```
{
  "resultCode": "SUCCESS",
  "message": "Destination set successfully",
  "data": {
```

```
    "rentId": 1001,  
    "destination": {  
      "x": 12.313,  
      "y": 32.3232  
    }  
  }  
}
```

❌ ROS2 → Server 응답 (실패: 잘못된 목적지 좌표)

```
{  
  "resultCode": "FAILURE",  
  "message": "Failed to set destination",  
  "errors": [  
    {  
      "field": "destination",  
      "message": "Invalid destination coordinates"  
    }  
  ]  
}
```